

이상 라벨링과 고장 임박 라벨링

IsolationForest 결과를 읽는 법부터 stratify 데이터 분리까지

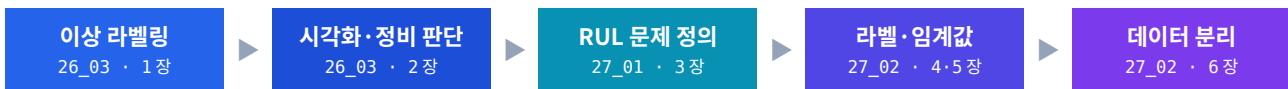
anomaly score_samples crosstab axhline RUL failure_soon 임계값 정답 누수 train_test_split stratify

2026년 9월 10일 (목) · 37 일차 · 80 차시

교안 26_03 이상 라벨링과 시각화 · 27_01 RUL 기반 고장 임박 문제 · 27_02 라벨 생성 및 데이터 준비

포스코 K-뉴딜 PDM 아카데미 · 박민호

오늘 다룬 것



오늘의 숫자	값	무엇인가
MIMII 소리 데이터	220 행 중 이상 40 건	IsolationForest contamination=0.18
교차표 채점	166 · 14 · 14 · 26	정상맞힘 · 거짓경보 · 놓침 · 이상맞힘
C-MAPSS 엔진 데이터	3,614 행 · 20 대	수명 147 ~ 215 cycle
failure_soon 라벨	620 / 2,994	1 이 17.2% — 클래스 불균형
학습용 / 평가용	2,891 / 723	test_size=0.2, stratify=y

연결

다음 시간 — 27_03 랜덤 포레스트 고장 임박 분류. 오늘 만든 `X_train·X_test·y_train·y_test` 네 덩어리를 그대로 받아 학습과 예측을 진행합니다.

오늘은 두 개의 이야기가 한 회차에 겹쳐 있습니다. 앞 절반은 어제 배운 IsolationForest의 출력을 실제로 쓸 수 있는 형태로 바꾸는 마무리 작업이고, 뒤 절반은 완전히 새로운 데이터(NASA 항공기 엔진)로 옮겨 가 **고장 임박 분류**라는 새 문제를 여는 출발점입니다. 그래서 1~2장과 3~6장은 다루는 데이터도, 목표도 다릅니다. 돌을 잇는 다리는 마지막 절에 따로 적어 두었습니다.

뱃지 — 이름 앞에 붙은 색 딱지

각 항목의 이름 왼쪽에 붙은 색 딱지는 그것이 문법적으로 무엇인지를 나타냅니다. 헛갈릴 때는 이렇게 판별합니다. **앞에 점(.)이 있나?** 없으면 함수나 키워드입니다. **점이 있으면 괄호()가 붙나?** 붙으면 메서드, 안 붙으면 속성입니다.

뱃지	뜻	오늘 나오는 예
함수	단독으로 호출하는 도구. 앞에 점이 없습니다.	<code>pd.crosstab()</code> · <code>train_test_split()</code>
메서드	값에 점을 찍고 괄호와 함께 부르는 도구.	<code>.astype()</code> · <code>.score_samples()</code>
문법	함수에 넘기는 옵션이나 표현 방식.	<code>stratify=y</code> · <code>cmap=</code>
개념	코드가 아니라 알고 있어야 할 생각의 틀.	정답 누수 · 클래스 불균형
패턴	여러 도구를 엮어 쓰는 정형화된 코드 뭉치.	조건 필터 후 점수 순 정렬

코드블록 — 회색 상자와 초록 하살표

회색 배경에 왼쪽이 파란 상자는 파이썬 코드입니다. 상자 안 아래쪽에 가로줄이 그어지고 **초록색** 표시가 나오면 그것은 제가 실제로 실행해 받은 출력입니다. 이 책에 실린 숫자는 전부 `26_mimii_features.csv`와 `27_cmapss_fd001_sample.csv`를 직접 돌려 확인한 값이고, 교안에 적힌 예상 결과를 옮긴 것이 아닙니다. 교안과 다른 값이 나온 곳은 부록 B에 대조표로 정리해 두었습니다.

오늘 처음 나오는 도구

아래 도구들은 오늘 처음 등장합니다. 본문에서 쓰기 전에 모두 정식으로 소개하니, 코드를 읽다 막히면 이 표에서 몇 장으로 가면 되는지 확인하십시오.

도구	한 줄 역할	처음 나오는 곳
<code>iso.score_samples()</code>	각 행이 얼마나 이상한지를 연속된 점수로 매깁니다. 작을수록 이상.	1 장
<code>Series.map()</code>	값 하나하나를 다른 값으로 바꿔치기합니다. 숫자 라벨을 글자 라벨로.	1 장
<code>pd.crosstab()</code>	두 컬럼을 행과 열로 교차시켜 개수를 셉니다. 채점표를 만드는 도구.	1 장
<code>plt.axhline()</code>	그래프에 가로 기준선을 긋습니다. 임계값을 그림에 박아 넣습니다.	2 장
<code>plt.colorbar()</code>	색의 진하기가 어떤 값에 대응하는지 알려 주는 막대를 붙입니다.	2 장
<code>plt.savefig()</code>	그린 그림을 이미지 파일로 저장합니다. 호출 순서에 함정이 있습니다.	2 장
<code>Series.nunique()</code>	서로 다른 값이 몇 개인지 셉니다. 설비 대수를 셀 때 씁니다.	3 장
<code>Series.astype(int)</code>	참/거짓을 1/0 정수로 바꿉니다. 라벨을 만드는 핵심 한 줄.	4 장
<code>Series.all()</code>	전부 참인지 한 번에 확인합니다. 누수 실험의 판정 도구.	5 장
<code>train_test_split()</code>	데이터를 학습용과 평가용 네 덩어리로 나눕니다.	6 장

지난 회차에서 이어지는 것

오늘 코드에는 이미 배운 도구가 그대로 다시 나옵니다. 소개는 반복하지 않고, 처음 다룬 회차만 적어 둡니다.

도구	오늘 하는 일	다룬 회차
<code>IsolationForest(contamination=.., random_state=42)</code>	오늘은 새로 배우지 않고, 그 출력을 쓸 수 있는 라벨로 바꾸는 데 집중합니다.	36 일차

도구	오늘 하는 일	다른 회차
<code>sns.scatterplot(..., hue=...)</code>	hue에 글자 라벨을 넘겨 정상과 이상을 색으로 갈라 그립니다.	26 일차
<code>plt.hist()</code>	이상 점수의 분포를 그려 봉우리와 꼬리를 확인합니다.	25 일차
<code>Series.value_counts(normalize=True)</code>	클래스 불균형을 개수와 비율 두 방식으로 확인합니다.	20 일차
<code>df.groupby(...)[...].max()</code>	설비별 최대 cycle을 구해 수명 길이로 씁니다.	20 일차
<code>df.info() · df.describe()</code>	새 데이터(C-MAPSS)의 규모와 센서 값 범위를 처음 훑습니다.	17 일차

오늘 다루는 두 데이터셋

앞뒤 절반이 서로 다른 데이터를 씁니다. 헛갈리지 않도록 둘을 나란히 놓습니다.

	MIMII 소리 특징	C-MAPSS 항공기 엔진
파일	26_mimii_features.csv	27_cmapss_fd001_sample.csv
크기	220 행 4 열	3,614 행 9 열
출처	설비가 내는 소리를 녹음해 특징 세 개로 요약한 것	NASA가 엔진 100대의 고장 과정을 시뮬레이션한 공개 데이터
입력	rms 세기 · spectral_centroid 톤 · zero_crossing_rate 거칠기	sensor_2 · 3 · 4 · 7 · 11 · 15 총 6개
정답	있음 — 채점용으로만 씁니다. 모델은 보지 않습니다.	없음 — RUL에서 직접 만들어 붙입니다.
하는 일	정답 없이 평소와 다른 것을 찾아냅니다.	정답을 만들어 놓고 그것을 맞히도록 가르칩니다.
쓰는 곳	1장 · 2장	3장 ~ 6장

CONTENTS

목차

오늘 배운 것의 지도

장	제목	무엇을 배우나
0 장	오늘의 지도	오늘 쓰이는 용어를 먼저 정리하고, 두 갈래 흐름을 한 장의 그림으로 잡습니다.
1 장	이상 라벨링 — 예측을 쓸 수 있는 정보로	모델이 받은 $-1/1$ 을 라벨로 바꿔 데이터에 붙이고, 점수를 매기고, 정답과 교차표로 채점합니다.
2 장	이상 시각화와 정비 판단	색 구분 산점도 · 기준선 · 색 농담 · 점수 히스토그램으로 결과를 보여 주고, 거짓 경보와 놓침의 균형을 정합니다.
3 장	RUL 과 고장 임박 문제	예지보전과 잔존 수명(RUL)의 개념을 잡고, NASA C-MAPSS 데이터를 처음 훑습니다.
4 장	파생 라벨과 임계값	$RUL \leq 30$ 규칙으로 <code>failure_soon</code> 을 만들고, 임계값을 바꿔 라벨이 어떻게 흔들리는지 실험합니다.
5 장	$X \cdot y$ 구성과 정답 누수	센서 6 개를 X 로, 정답을 y 로 나누고, RUL 을 반드시 빼야 하는 이유를 실험으로 확인합니다.
6 장	데이터 분리와 stratify	<code>train_test_split()</code> 의 세 옵션을 모두 적용해 학습용과 평가용을 나눕니다.
부록 A	치트시트	오늘 나온 도구를 한 표로 압축했습니다.
부록 B	오류 · 함정 사전	틀리기 쉬운 곳과, 교안 수치가 실행값과 어긋난 지점을 정리했습니다.
부록 C	실습 하이라이트 & 다음 시간 예고	오늘 실습에서 짚어 둔 것과 27_03 랜덤 포레스트 예고입니다.

CHAPTER 0 오늘의 지도

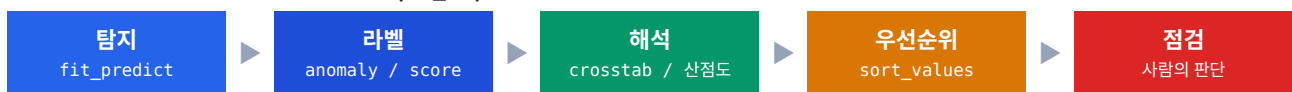
용어를 먼저 정리하고 시작합니다

오늘 배운 내용은 크게 둘로 나뉩니다. 앞쪽은 **어제 만든 이상탐지 모델의 뒷정리**입니다. 모델이 결과를 내놓았지만 그 결과는 아직 사람이 쓸 수 있는 모양이 아니었습니다. 그것을 라벨로 바꿔 붙이고, 그림으로 보여 주고, 정비 판단으로 잇는 것이 1~2 장입니다. 뒤쪽은 **완전히 새로운 문제의 시작**입니다. 소리 특징 데이터를 떠나 NASA 항공기 엔진 데이터로 옮겨 가서, 곧 고장 날 설비를 미리 골라내는 분류 문제를 세웁니다. 오늘은 모델을 학습시키는 데까지 가지 않고 **데이터를 모델이 먹을 수 있는 형태로 준비하는 데서 멈춥니다.**

0.1 오늘의 두 갈래

아래 두 줄이 오늘의 전부입니다. 첫 줄은 이미 만든 모델의 출력을 현장 행동까지 잇는 흐름이고, 둘째 줄은 새 문제를 세우는 6 단계 파이프라인 중 앞 네 단계입니다.

전반 — 이상탐지 결과를 정비 행동으로 (26_03)



후반 — 고장 임박 분류 6 단계 파이프라인 (27_01 · 27_02)



마지막 칸이 회색인 이유는 오늘 거기까지 가지 않았기 때문입니다. **오늘의 결승선은 네 번째 칸, 데이터를 네 덩어리로 나누는 것까지**입니다. 학습과 예측은 27_03 에서 다룹니다.

0.2 용어 정리

오늘 처음 나오는 말이 꽤 많습니다. 본문에서 다시 자세히 다루지만, 여기서 한 번 훑고 가면 읽기가 훨씬 수월합니다.

용어	한 줄 뜻	오늘 우리 데이터에서는
라벨(label)	각 행에 붙는 정답 또는 판정 꼬리표.	anomaly 0/1 · failure_soon 0/1
이상 점수	정도를 나타내는 연속된 숫자. 0/1 보다 정보가 많습니다.	score — 작을수록 이상
교차표	두 라벨을 행·열로 교차시켜 개수를 센 표. 채점표입니다.	pd.crosstab(label, anomaly)
거짓 경보	정상인데 이상이라고 잡은 것. 헛걸음 정비의 원인.	14 건
놓침	이상인데 정상이라고 넘긴 것. 돌발 정지의 원인.	14 건
예지보전(PdM)	달력이 아니라 설비의 실제 상태로 정비 시점을 정하는 방식.	이 과정 전체의 목표
RUL	잔존 수명. 고장까지 앞으로 더 버틸 수 있는 양.	0 ~ 214 cycle
cycle	설비가 한 번 작동한 운전 주기. 시·분·초가 아닙니다.	엔진당 147 ~ 215 회
파생 라벨	원래 없던 컬럼을 기존 컬럼에서 규칙으로 만들어 낸 정답.	RUL → failure_soon
임계값	0 과 1 을 가르는 경계 숫자. 데이터가 아니라 분석가가 정합니다.	RUL 30
클래스	분류에서 나뉜 각 묶음. 둘뿐이면 이진 분류입니다.	0 관촬음 · 1 곧 고장
클래스 불균형	한쪽 클래스가 다른 쪽보다 훨씬 많은 상태.	0 이 82.8% · 1 이 17.2%
feature (X)	모델에게 보여 주는 입력 단서들. 대문자 X 로 씁니다.	센서 6 개
target (y)	모델이 맞혀야 할 정답. 소문자 y 로 씁니다.	failure_soon
정답 누수	정답 정보가 입력에 섞여 드는 사고. 학습은 만점, 실전은 무력.	RUL 을 X 에 넣으면 발생

용어	한 줄 뜻	오늘 우리 데이터에서는
stratify	나눌 때 원래의 클래스 비율을 양쪽 모두에 유지시키는 옵션.	stratify=y

개념

라벨과 점수는 같은 결과의 두 얼굴입니다. 라벨은 이상이나 아니냐만 알려 주고, 점수는 얼마나 이상하냐를 알려 줍니다. 점검 목록을 만들 때는 라벨로 대상을 좁히고, 그 안에서 순서를 정할 때는 점수를 씁니다. 1장에서 두 가지를 모두 만드는 이유가 여기 있습니다.

0.3 다리 — 정답 없이 찾기에서 정답 만들어 맞히기로

1~2 장과 3~6 장 사이에는 데이터만 바뀌는 것이 아니라 **학습 방식 자체가 바뀝니다.** 이 둘의 차이를 잡아 두면 오늘 배운 것이 하나로 꿰입니다.

	비지도학습 — 1 장 · 2 장	지도학습 — 3 장 ~ 6 장
정답이	없습니다. 무엇이 이상인지 아무도 안 알려 줍니다.	있어야 합니다. 없으면 우리가 만듭니다.
모델이 하는 일	평소 무리에서 동떨어진 것을 찾아냅니다.	입력과 정답의 관계를 데이터에서 익힙니다.
오늘의 도구	IsolationForest	RandomForestClassifier (다음 시간)
결과의 성격	이건 평소와 다릅니다 — 점검 권유입니다.	이건 곧 고장입니다 — 정답을 맞힌 것입니다.
한계	왜 이상인지, 진짜 고장인지는 모릅니다.	정답이 붙은 사례가 충분히 많아야 합니다.

중요한 것은 **어느 쪽이 더 낫다는 이야기가 아니라는 점**입니다. 정답을 모을 수 없는 초기에는 비지도로 후보를 추리고, 점검 결과가 쌓여 정답이 생기면 지도학습으로 넘어갑니다. 오늘 1장에서 만든 라벨 표가 그대로 내일의 정답이 되는 구조입니다.

오늘 하지 않는 것

교안 목차에는 있지만 오늘 다루지 않은 것들입니다. 복습하다 안 나온다고 당황하지 마십시오.

안 한 것	언제
모델 학습과 예측 — fit · predict	다음 시간 (27_03)
정확도·재현율·정밀도 같은 성능 지표를 코드로 계산하기	다음 시간 (27_03)
RUL 숫자 자체를 맞히는 회귀	커리큘럼 마지막 단계
feature 를 고르는 기법	별도 주제 — 오늘은 준비된 6 개를 그대로 사용

CHAPTER 1 이상 라벨링

예측을 쓸 수 있는 정보로 바꾸기

어제 우리는 IsolationForest 로 이상탐지 모델을 만들었습니다. 그런데 그 모델이 내놓은 것은 -1 과 1 이 늘어선 배열 하나였습니다. 이 배열은 그 자체로는 점검에도, 보고에도, 시각화에도 바로 쓰기 어렵습니다. 몇 번 데이터가 이상한지 알려면 배열을 눈으로 훑어야 하고, 정비팀에 넘기려면 다시 표로 옮겨야 합니다.

오늘 1 장이 하는 일은 그 배열을 데이터 옆에 붙는 꼬리표로 바꾸는 것입니다. 꼬리표가 붙고 나면 조건 필터 한 줄로 이상만 골라낼 수 있고, 정렬 한 줄로 점검 순서를 만들 수 있고, 정답과 나란히 놓아 채점할 수도 있습니다. 강사가 표현한 대로 **비로소 쓸 수 있는 정보로 바뀌는 단계**입니다.

1.1 -1 과 1 을 0 과 1 로 뒤집기

먼저 짚어야 할 것은 **방향**입니다. IsolationForest 의 predict 는 이상을 -1, 정상을 1로 돌려줍니다. 우리가 쓸 라벨은 반대로 **이상인 1, 정상인 0** 입니다. 이상을 찾는 것이 목적이므로 찾고 싶은 쪽에 1을 주는 것이 관례이기 때문입니다. 그래서 단순히 자료형만 바꾸는 것이 아니라 값의 의미를 한 번 뒤집어야 합니다.

패턴 (pred == -1).astype(int) 모델 출력을 0/1 라벨로

정의 pred == -1 이 참/거짓 배열을 만들고, .astype(int)가 참을 1, 거짓을 0으로 바꿉니다. 이상이 1이 됩니다.

형태 df[새컬럼] = (pred == -1).astype(int)

어제 만든 모델을 그대로 다시 돌린 다음, 그 결과를 데이터에 붙여 보겠습니다. fit_predict()는 학습과 예측을 한 번에 하는 메서드입니다.

```
from sklearn.ensemble import IsolationForest

df = pd.read_csv("../Data/26_mimii_features.csv")
feats = ["rms", "spectral_centroid", "zero_crossing_rate"]

iso = IsolationForest(contamination=0.18, random_state=42)
pred = iso.fit_predict(df[feats])

# -1(이상) → 1, 1(정상) → 0    방향에 주의
df["anomaly"] = (pred == -1).astype(int)

print(int(df["anomaly"].sum()))

▶ 40
```

220 행 중 **40 행이 이상으로 잡혔습니다**. contamination=0.18을 주었으니 $220 \times 0.18 = 39.6$, 반올림해 40 개입니다. 이 숫자가 예상과 맞는지 세어 보는 것이 **방향이 뒤집히지 않았는지 확인하는 가장 빠른 방법**입니다. 만약 여기서 180 이 나왔다면 정상과 이상이 통째로 뒤바뀐 것입니다.

왜 쓰나 라벨이 0 과 1 이면 **합계가 곧 이상 건수**가 되고, 조건 필터에 그대로 쓸 수 있으며, 정답과 뺄셈·비교로 채점할 수 있습니다. 글자 라벨로는 이 중 어느 것도 편하게 되지 않습니다.

자주 하는 실수

(pred == 1).astype(int)로 쓰면 **정상이 1이 되어 모든 결과가 거꾸로 나옵니다**. 오류가 나지 않고 조용히 틀리기 때문에 가장 위험한 실수입니다. 라벨을 만든 직후 sum()으로 개수를 세어 contamination 비율과 맞는지 확인하는 습관을 들이십시오.

메서드 Series.map() 값을 다른 값으로 바꿔치기

정의 딕셔너리를 넘기면 키와 일치하는 값을 그 값으로 바꿔 새 Series를 돌려줍니다. 원본은 그대로입니다.

형태 시리즈.map({바꿀값: 새값, ...})

숫자 라벨은 계산에 좋지만 그림과 보고서에는 불친절합니다. 범례에 0 과 1 만 쓰면 어느 쪽이 이상인지 알 수 없습니다. 그래서 **글자 라벨을 하나 더 만들어 둡니다.**

```
df["label_name"] = df["anomaly"].map({1: "이상", 0: "정상"})
print(df["label_name"].value_counts().to_dict())

> {'정상': 180, '이상': 40}
```

이제 같은 판정이 두 벌 있습니다. 계산에는 `anomaly` 를, 그림과 보고에는 `label_name` 을 씁니다. 2 장에서 산점도를 그릴 때 `hue` 에 넘기는 것이 바로 이 글자 라벨입니다.

왜 쓰나 `hue="anomaly"` 로 그리면 범례가 **0** 과 **1** 로 나와 읽는 사람이 해석해야 합니다. `hue="label_name"` 이면 범례에 바로 **정상·이상**이 뜹니다. 그림 한 장을 남에게 넘길 때 이 차이가 큼니다.

주의

`map()`에 없는 값이 들어오면 오류가 아니라 `NaN` 이 됩니다. 위 **딕셔너리**에 **0** 과 **1** 만 적었는데 **라벨**에 **2** 가 섞여 있었다면 그 행은 **조용히 결측**이 됩니다. 변환 후 `value_counts()`로 개수 합이 원래 행 수와 같은지 확인하십시오.

1.2 얼마나 이상한가 — 점수 매기기

여기부터가 오늘 강의에서 **가장 많이 언급된 대목**입니다. 라벨은 이상이나 아니냐만 알려 줍니다. 그런데 이상으로 잡힌 40 개를 한 번에 점검할 수는 없습니다. **무엇부터 볼 것인가**라는 질문에 답하려면 정도를 나타내는 연속된 숫자가 필요합니다. 그것이 이상 점수입니다.

메서드 `iso.score_samples()` 각 행이 얼마나 이상한지

정의 각 행에 대해 이상 점수를 돌려줍니다. **작을수록(더 음수일수록) 더 이상합니다.** 0/1 라벨과 달리 정도가 연속적으로 표현됩니다.

형태 `iso.score_samples(df[특징컬럼들])`

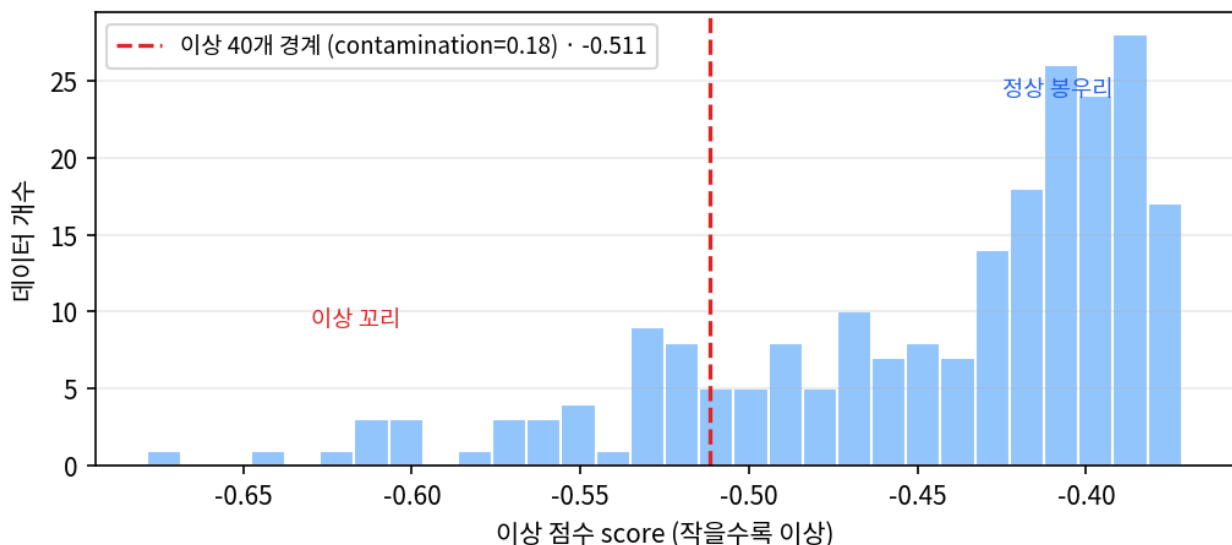
라벨을 만들었던 그 모델 객체에 그대로 물어보면 됩니다. 다시 학습시킬 필요가 없습니다.

```
df["score"] = iso.score_samples(df[feats])

print(round(float(df["score"].min()), 3), round(float(df["score"].max()), 3))

> -0.678 -0.372
```

점수는 **-0.678**에서 **-0.372** 사이에 퍼져 있습니다. 전부 음수라서 처음 보면 당황스러운데, 이 알고리즘에서는 값의 절대적인 크기가 아니라 **다른 행과 비교했을 때 얼마나 낮은가**가 의미를 가집니다. 아래 그림이 그 분포입니다.



▲ 이상 점수의 분포 — 오른쪽에 정상 봉우리가 두툼하게 서 있고 왼쪽으로 이상 꼬리가 길게 늘어집니다. 빨간 점선이 contamination=0.18 이 그른 40 개째 경계(-0.511)입니다

왜 쓰나 라벨만 있으면 40 개가 모두 똑같이 이상해 보입니다. 점수가 있으면 **가장 먼저 봐야 할 한 대**를 짚을 수 있습니다. 정비 인력은 한정돼 있으므로, 실무에서 실제로 쓰이는 것은 라벨보다 이 순서입니다.

응용 히스토그램은 contamination 을 얼마로 둘지 가늠하는 근거도 됩니다. 봉우리와 꼬리 사이가 뚜렷하게 갈라지면 그 골짜기가 자연스러운 경계이고, 위 그림처럼 완만하게 이어지면 **어디를 잘라도 애매한 행이 남는다**는 뜻입니다. 그 애매한 구간이 곧 거짓 경보와 놓침이 생기는 자리입니다.

개념

score_samples()와 decision_function()은 다릅니다. decision_function()은 경계를 0 에 맞춰 놓아 **음수면 이상**으로 바로 읽히고, score_samples()는 그 기준선만큼을 빼지 않은 원점수입니다. 오늘 코드는 score_samples()를 쓰므로 값이 전부 음수로 나오는 것이 정상입니다.

패턴 조건 필터 + sort_values() 점검 우선순위 목록 만들기

정의 이상인 행만 골라낸 뒤 점수가 작은 순으로 정렬하면 그 표가 곧 **점검 순서**가 됩니다.

형태 df[df["anomaly"] == 1].sort_values("score")

220 행을 40 행으로 좁히고, 그 40 행에 순서를 매기는 두 단계입니다.

```
anom_df = df[df["anomaly"] == 1].sort_values("score")

print(len(anom_df))
print(anom_df[feats + ["score"]].head(5).round(4).to_string())
```

	rms	spectral_centroid	zero_crossing_rate	score
▶ 40				
▶ 70	0.1599	1947.0	0.0891	-0.6785
▶ 54	0.0144	1864.1	0.1384	-0.6388
▶ 108	0.1467	2674.4	0.1037	-0.6253
▶ 157	0.1273	1577.0	0.0811	-0.6136
▶ 149	0.1303	2126.7	0.1419	-0.6109

1 순위는 **70 번**입니다. rms 가 0.1599 로 전체에서 가장 큰 값이니 **소리가 유난히 큰 이상**입니다. 그런데 2 순위인 54 번을 보십시오. rms 가 0.0144 로 오히려 전체에서 **가장 작습니다**. 이 두 건은 성격이 정반대인데 둘 다 이상으로 잡혔습니다.

왜 쓰나 이 표를 그대로 정비팀에 넘길 수 있습니다. 220 개를 보라는 말은 아무도 실행할 수 없지만, **점수 순으로 정렬된 40 개**는 위에서부터 차례로 처리하면 되는 지시가 됩니다.

개념

IsolationForest 는 **너무 큰 값뿐 아니라 너무 작은 값도 이상으로 잡습니다**. 평소 무리에서 어느 방향으로든 동떨어져 있으면 걸립니다. 2 장에서 다룰 rms 기준선이 큰 쪽만 잡아내는 것과 대비되는 지점입니다.

1.3 정답과 맞춰 보기 — 교차표

지금 쓰는 데이터에는 label 이라는 정답 컬럼이 있습니다. 실제 현장에는 없는 것이지만, 연습용 데이터에는 붙어 있으니 **모델이 얼마나 맞혔는지** 채점해 볼 수 있습니다.

함수 pd.crosstab() 두 라벨을 교차시켜 개수 세기

정의 첫 인자를 행, 둘째 인자를 열로 두고 **각 조합에 몇 개가 들어가는지** 센 표를 만듭니다.

형태 pd.crosstab(정답컬럼, 모델라벨컬럼)

```
ct = pd.crosstab(df["label"], df["anomaly"])
print(ct)
```

```
▶ anomaly    0    1
▶ label
▶ 0           166   14
▶ 1           14   26
```

네 칸을 하나씩 읽어야 의미가 생깁니다. 아래 표가 각 칸이 현장에서 무엇을 뜻하는지 옮긴 것입니다.

왜 쓰나 맞은 개수만 세면 220 개 중 192 개를 맞춰 **87.3%**라는 좋아 보이는 숫자가 나옵니다. 그런데 이 데이터는 정상이 180 개라 **전부 정상이라고 답만 해도 81.8%**입니다. 교차표를 봐야 그 착시가 걷힙니다.

칸	개수	현장에서의 뜻	비용
정상 맞힘	166	정상인 것을 정상이라고 넘겼습니다. 아무 일도 일어나지 않습니다.	없음
거짓 경보	14	정상인데 이상이라고 잡았습니다. 정비사가 가 보면 멀쩡합니다.	헛걸음 · 피로
놓침	14	이상인데 정상이라고 넘겼습니다. 그대로 돌아 멈춥니다.	돌발 정지
이상 맞힘	26	이상인 것을 제대로 잡았습니다. 모델이 값을 하는 칸입니다.	—

정답 이상 40 건 중 **26 건을 잡고 14 건을 놓쳤습니다(재현율 65.0%)**. 이상이라고 지목한 40 건 중 실제 이상은 26 건이었습니다(정밀도 65.0%). 어제 배운 Z-score 방식이 잡은 것이 23 건이었으니 **26 건은 분명한 개선**이지만, 절반을 조금 넘겼을 뿐입니다. 놓친 14 건은 정상 무리 안에 섞여 있어 세 특징만으로는 갈라낼 수 없었던 사례입니다.

주의

교안 슬라이드에는 이 표가 **167 · 13 · 27**로 적혀 있습니다. 같은 파일을 같은 옵션으로 돌리면 **166 · 14 · 14 · 26**이 나옵니다. 이 책에는 실행값을 실었고, 대조표는 부록 B에 있습니다. 교안의 예상 결과와 내 실행 결과가 다르면 **먼저 의심할 것은 내 코드지만, 확인 뒤에는 실행값을 믿으십시오.**

설비 적용

채점은 **정답이 붙어 있는 연습 데이터에서만 가능합니다**. 현장에는 이 설비가 진짜 고장이었는지 알려 주는 컬럼이 없습니다. 그래서 실무에서는 모델이 지목한 것을 사람이 점검하고, 그 결과를 다시 쌓아 정답을 만들어 갑니다. **오늘 만든 표가 내일의 정답이 되는 구조**입니다.

숫자와 표는 정확하지만 **이상이 어디에 모여 있는지는** 알려 주지 않습니다. 앞 장에서 만든 40 개 목록을 그대로 보여 주면 받는 사람은 숫자를 하나씩 읽어야 합니다. 반면 점 220 개를 찍고 색을 두 가지로 갈라 놓으면, 무리에서 떨어진 빨간 점들이 **보자마자** 눈에 들어옵니다. 이 장은 그 그림을 만드는 방법과, 그 그림을 정비 판단으로 옮기는 방법을 다룹니다.

2.1 색으로 갈라 그리는 산점도

함수 `sns.scatterplot(..., hue=...)` 라벨에 따라 색이 갈리는 산점도

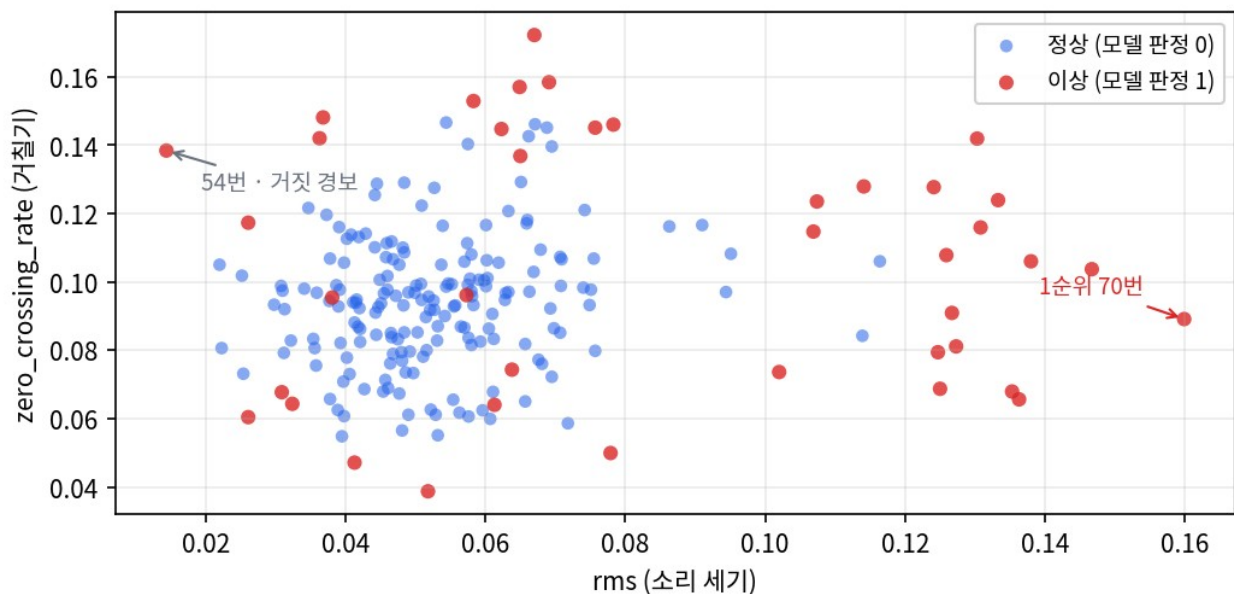
정의 x와 y에 특징 하나씩을 주고, hue에 라벨 컬럼을 주면 **색 구분과 범례가 자동으로** 만들어집니다.

형태 `sns.scatterplot(data=df, x="컬럼", y="컬럼", hue="라벨컬럼", alpha=0.6, s=40)`

산점도 자체는 26 일차에 다뤘습니다. 오늘 새로운 것은 hue에 **1 장에서 만든 글자 라벨을** 넘긴다는 점입니다.

```
sns.scatterplot(data=df, x="rms", y="zero_crossing_rate",
                hue="label_name", alpha=0.6, s=40)
plt.title("정상 vs 이상 산점도")
plt.xlabel("rms (소리 세기)")
plt.show()
```

그림에서 세 가지가 한 번에 읽힙니다. 첫째, 오른쪽 끝에 **rms가 큰 이상**이 확실하게 떨어져 나가 있습니다. 둘째, 왼쪽 끝의 54 번은 rms가 오히려 가장 작는데도 이상으로 잡혔습니다. 셋째, 그리고 가장 중요한 것 — **가운데 정상 무리 안에 빨간 점이 여러 개 박혀 있습니다.** 이 자리가 회색 지대이고, 앞 장에서 본 거짓 경보 14 건이 대부분 여기서 나옵니다.



▲ 가로축 소리 세기, 세로축 거칠기 — 오른쪽으로 멀리 나가 무리가 세기 이상입니다. 파랑과 빨강이 섞인 가운데 구역이 거짓 경보와 놓침이 생기는 회색 지대입니다

왜 쓰나 hue에 **글자 라벨**을 주는 이유가 여기서 드러납니다. 숫자 라벨을 주면 범례가 0과 1로 뜨는데, 그림을 넘겨받은 사람은 어느 쪽이 이상인지 다시 물어봐야 합니다.

응용 산점도는 한 번에 두 특징만 보여 줍니다. 그래서 **조합을 바꿔 가며 여러 장을 그려야** 이상의 성격이 드러납니다. rms가 축에 들어간 조합에서는 세기가 큰 이상이 잘 보이고, rms를 뺀 조합에서는 세기는 평범한데 톤과 거칠기의 짝이 이상한 다변량 이상이 드러납니다.

```

combos = [("rms", "zero_crossing_rate"),
          ("rms", "spectral_centroid"),
          ("spectral_centroid", "zero_crossing_rate")]

for x, y in combos:
    sns.scatterplot(data=df, x=x, y=y, hue="label_name")
    plt.title(f"{x} vs {y}")
    plt.show()

```

주의

alpha는 투명도입니다. 점이 겹치는 곳이 진해지므로 **정상 무리의 밀도**가 드러납니다. 220 개 정도면 `alpha=0.6`, `s=40` 이 적당하고, 점이 수천 개로 늘면 alpha를 더 낮추십시오. 한글 폰트를 설정하지 않으면 제목과 축 이름이 전부 네모(□□□)로 나옵니다.

2.2 기준선 — 경계를 그림에 직접 긋기

임계값 0.135 라고 글로 쓰는 것과, 그래프에 가로선 하나를 긋는 것은 전달력이 전혀 다릅니다. **선 위로 솟은 점이 곧 의심 대상**이라는 사실이 설명 없이 읽힙니다.

함수 `plt.axhline()` 가로 기준선 긋기

정의 y로 지정한 높이에 그래프 전체를 가로지르는 선을 그립니다. 세로선은 `plt.axvline()`입니다.

형태 `plt.axhline(y=값, color="red", linestyle="--", label="설명")`

25 일차에 배운 평균+3 시그마 임계값을 그대로 선으로 옮겨 보겠습니다. 가로축은 특별한 의미 없이 **데이터 순서**를 씁니다.

```

thr = df["rms"].mean() + 3 * df["rms"].std()
print(round(thr, 3))

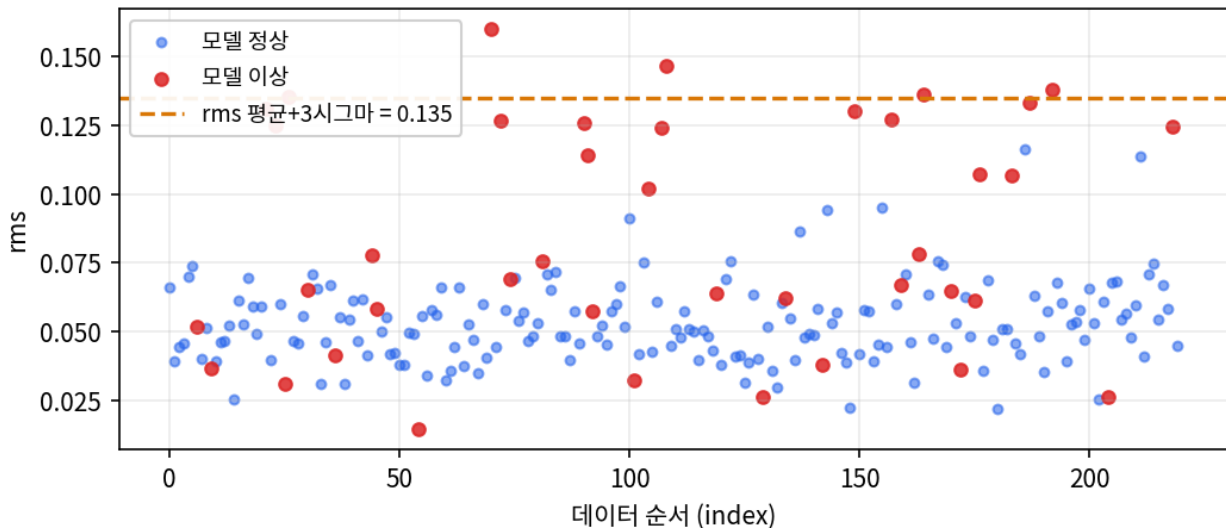
plt.scatter(range(len(df)), df["rms"], s=12)
plt.axhline(y=thr, color="red", linestyle="--", label="평균+3 시그마 기준선")
plt.legend()
plt.show()

print(int((df["rms"] > thr).sum()))

```

▶ 0.135
▶ 5

기준선은 **0.135** 이고, 그 위로 솟은 점은 **5 개**뿐입니다. 앞 장에서 IsolationForest 가 잡은 이상은 40 개였는데 말입니다.



▲ 빨간 점이 모델이 이상으로 잡은 40 개 — 그중 기준선(주황 점선) 위에 있는 것은 5 개뿐이고 나머지 35 개는 선 아래에 숨어 있습니다

왜 쓰나 그림에 선을 그으면 **선을 어디에 긋느냐에 따라 무엇이 걸리고 무엇이 빠지는지**가 눈에 보입니다. 임계값을 조정하는 회의에서 이 그림 한 장이 표 열 개보다 빠릅니다.

응용 이 그림의 진짜 교훈은 **단일 특징 기준선의 한계**입니다. rms 하나로 그은 선은 세기가 큰 이상 5 건만 잡아냅니다. 나머지 35 건은 세기는 평범한데 톤이나 거칠기와의 조합이 이상한 경우라 선 아래에 그대로 숨습니다. 여러 특징을 한꺼번에 보는 모델이 필요한 이유가 이 한 장에 있습니다.

설비 적용

설비 데이터는 대개 시간 순서가 있으므로, 가로축에 순서 대신 **시간**을 두면 훨씬 강력해집니다. 이상이 후반부에 몰려 있다면 그것은 점점 나빠지고 있다는 뜻이고, 고르게 흩어져 있다면 산발적인 잡음일 가능성이 큼니다. 같은 개수의 이상이라도 **언제 났는가**에 따라 대응이 달라집니다.

문법 `c= · cmap= · plt.colorbar()` 점수를 색의 진하기로

정의 scatter의 c에 숫자 컬럼을 주면 값에 따라 색 진하기가 자동으로 매겨지고, colorbar()가 색과 값의 대응표를 옆에 붙입니다.

형태 `sc = plt.scatter(x, y, c=점수컬럼, cmap="viridis", s=30) ; plt.colorbar(sc, label="설명")`

정상과 이상 두 색으로 나누면 경계 부근의 애매한 행들이 어느 한쪽으로 강제로 밀려납니다. 점수를 색 농담으로 표현하면 **그 애매함이 그대로 보입니다**.

```
sc = plt.scatter(df["rms"], df["zero_crossing_rate"],
                 c=df["score"], cmap="viridis", s=30)
plt.colorbar(sc, label="이상 점수 (작을수록 이상)")
plt.title("이상 점수 색 농담 산점도")
plt.show()
```

왜 쓰나 두 색 그림은 **판정**을 보여 주고, 색 농담 그림은 **정도**를 보여 줍니다. 회색 지대가 실제로 얼마나 넓은지 확인하려면 후자가 필요합니다.

주의

colorbar()에는 scatter()가 돌려준 객체를 넘겨야 합니다. `sc` =로 받아 두지 않고 `plt.colorbar()`만 부르면 어느 그림의 색인지 몰라 오류가 나거나 엉뚱한 막대가 붙습니다.

함수 `plt.savefig()` 그림을 파일로 저장

정의 그린 그림을 이미지 파일로 저장합니다. 반드시 `plt.show()`보다 먼저 **불러야 합니다**.

형태 `plt.savefig("파일명.png", dpi=150, bbox_inches="tight")`

```
sns.scatterplot(data=df, x="rms", y="zero_crossing_rate", hue="label_name")
plt.title("정상 vs 이상 산점도")

plt.savefig("anomaly_scatter.png", dpi=150, bbox_inches="tight") # 먼저
plt.show() # 나중
```

왜 쓰나 보고서나 메신저로 그림을 넘기려면 파일이 있어야 합니다. dpi=150 이면 문서에 넣어도 흐리지 않고, bbox_inches="tight"는 축 이름이 잘리는 것을 막아 줍니다.

자주 하는 실수

plt.show()를 먼저 부르면 빈 그림이 저장됩니다. show()가 화면에 그린 뒤 캔버스를 비우기 때문입니다. 오류도 나지 않고 파일도 멀쩡히 생기는데 열어 보면 하얗습니다. 초보자가 가장 자주 겪는 함정이니 순서를 몸에 익히십시오.

2.3 거짓 경보와 놓침, 무엇을 감수할 것인가

앞 장의 교차표에서 거짓 경보 14 건과 놓침 14 건이 나왔습니다. 우연히 같은 숫자지만 **두 실수의 비용은 전혀 다릅니다**. 거짓 경보는 정비사가 헛걸음하는 것으로 끝나고, 놓침은 설비가 돌다가 멈추는 것으로 끝납니다. 어느 쪽을 줄일지는 설비마다 다르게 정해야 합니다.

설비 성격	치명적인 실수	경계를 어떻게	실제 조정
핵심 설비 — 멈추면 라인 전체가 섬	놓침	민감하게. 거짓 경보를 감수하고 많이 잡습니다.	contamination 올림
보조 설비 — 여분이 있음	거짓 경보	둔감하게. 정비사의 피로를 줄이는 쪽으로.	contamination 내림

같은 모델, 같은 데이터라도 **설비 중요도에 따라 다이얼을 다르게 돌리는 것이 맞습니다**. 정답이 하나가 아니라는 뜻입니다. 현장 피드백을 보면서 경보가 너무 잦은지 너무 드문지 조정해 나가고, 최종적으로는 **경보가 울리면 대체로 진짜더라**는 신뢰를 쌓는 방향으로 맞춥니다.

2.4 이상탐지 리포트에 담을 것

오늘 만든 것들을 한 장으로 묶으면 리포트가 됩니다. 받는 사람이 **무엇을, 왜, 어떤 순서로** 점검할지 바로 알 수 있어야 좋은 리포트입니다.

구성	내용	오늘 데이터로 쓰면
요약 한 줄	전체 건수 · 이상 건수 · 1 순위	전체 220 건 중 이상 40 건, 1 순위 70 번 — 점검 권유
그림 한 장	이상이 어디에 모였는지 보이는 산점도	2.1 의 색 구분 산점도에 기준선을 얹은 것
우선순위 표	점수 순으로 정렬한 상위 몇 건	70 · 54 · 108 · 157 · 149 번
한계 한 줄	모델이 무엇을 못 하는지	놓칠 수 있으므로 사람 점검을 병행 · 채점은 학습 데이터에서만 가능

개념

이상탐지가 잡은 것은 고장이 아니라 평소와 다름입니다. 운전 조건이 바뀌었거나 센서가 흔들렸을 때도 똑같이 걸립니다. 그래서 리포트는 고장 확정이 아니라 **점검 권유**로 써야 합니다. 최종 판단은 데이터가 아니라 정비 지식을 가진 사람이 합니다.

여기서 데이터가 바뀝니다. 어제부터 오늘 2 장까지 쓰던 것은 설비가 내는 **소리의 특징**이었습니다. 3 장부터는 **NASA 가 공개한 항공기 엔진 데이터(C-MAPSS FD001)**를 씁니다. 목표도 바뀝니다. 지금까지는 정답 없이 평소와 다른 것을 찾아내는 일이었고, 지금부터는 **정답을 만들어 놓고 그것을 맞히도록 가르치는** 일입니다.

오늘 3~6 장에서는 **모델을 학습시키지 않습니다**. 문제를 정의하고, 정답 라벨을 만들고, 입력과 정답을 나누고, 학습용과 평가용으로 쪼개는 데까지가 전부입니다. 강사의 표현대로 **데이터를 모델이 먹을 수 있는 형태로 차려 놓는 단계**입니다.

3.1 잔존 수명(RUL)이라는 숫자

개념 RUL (Remaining Useful Life) 고장까지 앞으로 더 버틸 수 있는 양

정의 잔존 수명. **RUL 이 작아질수록 고장이 임박했다**는 뜻입니다. RUL 50 이면 앞으로 50 만큼 더 작동한 뒤 고장난다는 의미입니다.

형태 $RUL = \text{최종 고장 cycle} - \text{현재 cycle}$

단위가 시·분·초가 아니라 **cycle**이라는 점이 낯섭니다. cycle 은 설비가 한 번 작동한 운전 주기입니다. 엔진이라면 한 번의 비행, 프레스라면 한 번의 타격입니다. 시간으로 세는 것보다 **실제로 얼마나 썼는가**를 더 정확히 반영합니다.

왜 쓰나 RUL 은 정비 시점을 정하는 **가장 직접적인 근거**입니다. 모든 설비의 RUL 을 알면 작은 것부터 점검하면 되므로, 숫자 하나로 우선순위가 정해집니다.

개념

마라톤으로 생각하면 쉽습니다. 달린 거리가 cycle, 남은 거리가 RUL 입니다. 한 걸음 더 달릴수록 남은 거리는 정확히 한 걸음 줄어듭니다. 그래서 **같은 설비 안에서는 cycle 이 1 늘면 RUL 이 정확히 1 줄어듭니다**.

3.2 데이터 첫 만남

새 데이터를 받으면 늘 하던 순서대로 훑습니다. 크기를 보고, 자료형과 결측을 보고, 값의 범위를 봅니다. 17 일차에 배운 `info()`와 `describe()`가 그대로 쓰입니다.

```
import pandas as pd

df = pd.read_csv("../Data/27_cmapss_fd001_sample.csv", encoding="utf-8")

print(df.shape)
print(df.columns.to_list())
print("결측 합계:", int(df.isna().sum().sum()))

> (3614, 9)
> ['unit_id', 'cycle', 'sensor_2', 'sensor_3', 'sensor_4', 'sensor_7', 'sensor_11',
'sensor_15', 'RUL']
> 결측 합계: 0
```

3,614 행 9 열, 결측치는 하나도 없습니다. 컬럼 9 개는 성격이 셋으로 갈리는데, 이 구분이 5 장에서 X와 y 를 나눌 때 결정적으로 쓰입니다. 지금 눈에 익혀 두십시오.

류음	컬럼	역할	나중에
식별	unit_id · cycle	어느 설비의 몇 번째 시점인지 알려 주는 주소입니다.	X 에서 제외
관찰	sensor_2 ~ sensor_15	설비 상태를 보여 주는 센서값 6 개입니다.	X 가 됨
결과	RUL	맞히고 싶은 값. 여기서 정답 라벨을 파생시킵니다.	X 에서 제외

한 행은 한 설비가 한 운전 시점에 보인 상태입니다. 같은 unit_id가 여러 행에 반복해서 나오고, 200 cycle을 산 엔진은 약 200 줄을 차지합니다. 표를 가로로 읽으면 그 순간의 컨디션이고, 같은 설비의 행을 세로로 이어 읽으면 시간에 따른 변화의 이야기가 됩니다.

메서드 Series.nunique() 서로 다른 값이 몇 개인지

정의 중복을 제외하고 고유한 값의 개수를 셉니다. 값 자체가 아니라 종류의 수를 돌려줍니다.

형태 df["컬럼"].nunique()

```
print("설비 대수:", df["unit_id"].nunique())
print("전체 행 수:", len(df))
```

- ▶ 설비 대수: 20
- ▶ 전체 행 수: 3614

엔진 20대의 기록이 3,614 행에 섞여 있습니다. 한 대당 평균 181 행입니다.

왜 쓰나 20대가 뒤섞인 채로 통계를 내면 아무 의미가 없습니다. 평균 RUL을 구해 봐야 수명이 제각각인 엔진들의 평균일 뿐입니다.

설비별로 그룹을 지어 보거나 한 대만 떼어 봐야 제대로 된 분석이 됩니다.

연결

nunique()는 값의 종류를 세고, 20 일차에 배운 value_counts()는 각 값이 몇 번 나왔는지를 셉니다.

df["unit_id"].value_counts()를 쓰면 엔진마다 몇 행씩 있는지, 즉 각 엔진이 몇 cycle을 살았는지가 나옵니다.

3.3 설비 한 대의 일생, 그리고 설비끼리의 비교

패턴 조건 필터 + groupby().max() 한 대만 떼어 보기 · 설비별 수명 구하기

정의 조건 필터로 설비 한 대를 골라 내고, groupby로는 설비별 최대 cycle을 구합니다. 최대 cycle이 곧 그 설비의 수명입니다.

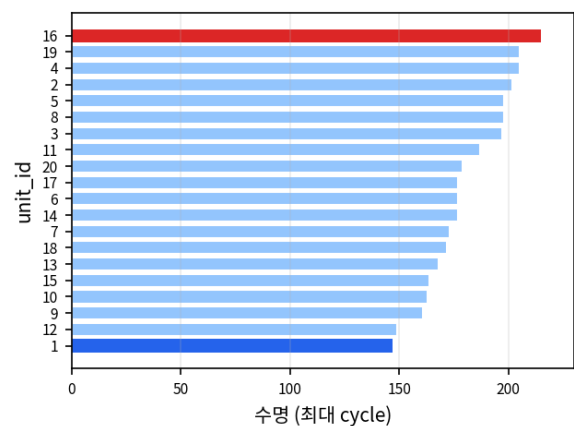
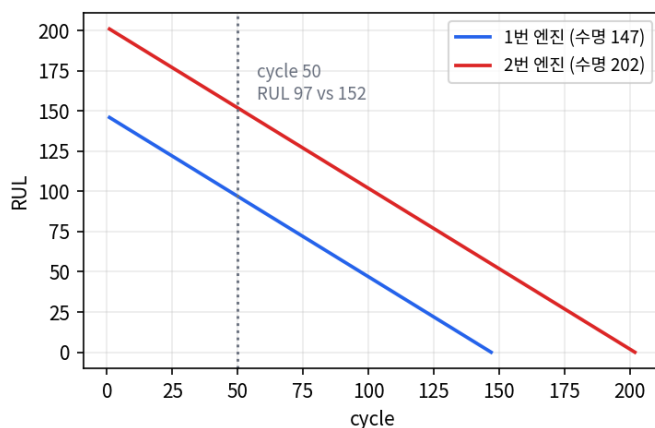
형태 df[df["unit_id"] == 1] · df.groupby("unit_id")["cycle"].max()

```
unit1 = df[df["unit_id"] == 1]
print("1번 엔진 행 수:", len(unit1))
```

```
life = df.groupby("unit_id")["cycle"].max().sort_values()
print("최단 수명:", life.index[0], "번 엔진", life.iloc[0], "cycle")
print("최장 수명:", life.index[-1], "번 엔진", life.iloc[-1], "cycle")
```

- ▶ 1번 엔진 행 수: 147
- ▶ 최단 수명: 1번 엔진 147 cycle
- ▶ 최장 수명: 16번 엔진 215 cycle

같은 종류의 엔진 20대인데 147 cycle에서 215 cycle까지, 1.5배 가까이 차이가 납니다. 여기서 오늘 3장의 가장 중요한 함정이 나옵니다.



▲ 왼쪽 — 두 엔진의 RUL 은 둘 다 우하향 직선이지만 출발 높이가 다릅니다. 같은 50 cycle 시점에 1 번은 RUL 97, 2 번은 152 입니다. 오른쪽 — 20 대의 수명이 147 부터 215 까지 흩어져 있습니다

왜 쓰나 모델에 넣을 입력을 고를 때 이 그림이 근거가 됩니다. cycle 은 설비마다 의미가 달라 **그대로는 비교할 수 없는 값**이라는 사실이 여기서 확인됩니다.

응용 같은 50 cycle 시점을 두 엔진에서 각각 꺼내 보면 차이가 숫자로 드러납니다.

```
print(df[(df["unit_id"] == 1) & (df["cycle"] == 50)][["unit_id", "cycle", "RUL"]])
print(df[(df["unit_id"] == 2) & (df["cycle"] == 50)][["unit_id", "cycle", "RUL"]])
```

	unit_id	cycle	RUL
▶	49	1	50
▶			
▶	196	2	50

둘 다 50 번째 운전인데 1 번 엔진은 앞으로 97, 2 번 엔진은 152 가 남았습니다. **50** 이라는 숫자만 보고는 어느 쪽이 더 위험한지 알 수 없습니다.

자주 하는 실수

cycle 이 1 늘면 RUL 이 1 준다는 규칙은 같은 unit_id 안에서만 성립합니다. 설비끼리는 고장 시점이 다르므로 cycle 만으로 비교하면 안 됩니다. 5 장에서 cycle 을 입력에서 빼는 이유가 바로 이것입니다.

3.4 사람이 규칙을 적는 방식의 한계

모델을 쓰기 전에, 왜 굳이 모델이 필요한지부터 짚고 갑니다. 가장 단순한 방법은 사람이 직접 기준을 적는 것입니다. 온도가 90 도를 넘으면 위험처럼 말합니다. 이해하기 쉽고 설명하기도 좋습니다. 그런데 강의에서 세 가지 질문이 던져졌고, 셋 다 답하기가 어렵습니다.

질문	왜 답하기 어려운가
온도 하나로 충분한가	고장은 온도 하나가 아니라 진동·압력·회전 속도 등 여러 값이 얹혀 일어납니다. 하나만 봐서는 놓칩니다.
88 도는 안전하고 90 도는 위험한가	그 경계를 누가 정하니까. 아무도 모릅니다. 경험과 직관에는 한계가 있습니다.
변수가 스물한 개면	두 개짜리 규칙도 경계를 못 정하는데, 스물한 개의 조합마다 규칙을 손으로 적는 것은 사실상 불가능합니다.

오늘 우리 데이터로 확인해 보면 이 말이 실감납니다. 센서 여섯 개의 값 범위가 서로 완전히 다릅니다.

```
print(df[feats].describe().T[["mean", "std", "min", "max"]].round(3).to_string())
```

	mean	std	min	max
▶ sensor_2	642.716	0.717	640.384	645.006
▶ sensor_3	1588.630	4.817	1572.759	1604.662
▶ sensor_4	1408.919	9.056	1379.889	1437.818
▶ sensor_7	553.125	0.832	550.291	555.201
▶ sensor_11	47.559	0.367	46.447	48.753
▶ sensor_15	8.474	0.067	8.285	8.696

sensor_3 는 1,588 근처에서 놓고 sensor_15 는 8.47 근처에서 놓습니다. **평균이 200 배 차이 나고, 표준편차도 0.067 에서 9.056 까지 벌어집니다.** 이런 여섯 개를 놓고 사람이 손으로 조합 규칙을 적는 것은 현실적이지 않습니다. 각각의 경계를 정하는 것도 어렵거니와, 두 센서가 동시에 어떤 방향으로 움직일 때 위험한지는 더더욱 알 수 없습니다.

머신러닝은 그 규칙을 **사람이 적는 대신 데이터에서 찾아내게 하는 방식**입니다. 정답이 붙은 사례를 충분히 보여 주면, 모델이 자기 답과 정답을 비교하며 기준을 조금씩 고쳐 나갑니다. 그 반복이 끝나면 처음 보는 데이터도 꽤 맞게 됩니다.

개념

학습의 목표는 **외우기가 아니라 일반화**입니다. 본 문제만 잘 푸는 것은 실력이 아닙니다. 그래서 6장에서 데이터를 학습용과 평가용으로 나누고, 처음 보는 데이터에서 몇 점을 받는지로 실력을 잽니다. 사례가 적으면 엉뚱한 규칙을 만들기 쉬우므로, **좋은 데이터를 충분히 모으는 일**이 알고리즘을 고르는 일만큼 중요합니다.

연결

이 모든 계산을 직접 구현하지는 않습니다. **scikit-learn** 이라는 라이브러리가 이미 만들어 두었기 때문입니다. 요리할 때 칼과 냄비를 직접 만들지 않고 사 와서 쓰는 것과 같습니다. 우리가 할 일은 **무엇을 하려는지 정하고 재료를 제대로 손질하는 것**이고, 복잡한 계산은 라이브러리가 맡습니다. 오늘 쓴 `train_test_split` 도, 다음 시간에 쓸 `RandomForestClassifier` 도 전부 여기서 나옵니다.

3.5 회귀나 분류나 — 문제를 다시 쓰기

이제 무엇을 맞힐지 정해야 합니다. 같은 데이터로도 질문을 두 가지로 세울 수 있고, **오늘 수업에서 가장 오래 다룬 대목이 바로 이 갈림길**입니다.

구분	회귀 (Regression)	분류 (Classification)
질문	앞으로 얼마나 남았나	곧 고장인가 아닌가
출력	연속된 숫자 — RUL 47	둘 중 하나 — 0 또는 1
장점	정보가 풍부해 정비 계획을 정밀하게 세울 수 있습니다.	결과를 바로 행동으로 옮길 수 있고 평가가 단순합니다.
난이도	높습니다. 정답 경우의 수가 사실상 무한합니다.	낮습니다. 맞았는지 한 칸씩 비교하면 끝납니다.
오늘의 선택	나중에 — 커리큘럼 마지막 단계	이쪽입니다

분류를 먼저 하는 이유는 실력이 부족해서가 아니라 **순서가 그렇기 때문**입니다. 분류로 불러오기부터 예측까지 전체 흐름을 한 번 완성해 보면 큰 틀이 손에 익습니다. 그 토대 위에서 회귀로 넘어가는 것이 훨씬 수월합니다. 게다가 현장에서는 정밀한 숫자보다 **당장 점검할 것이냐 아니냐**라는 답이 더 자주 필요합니다.

개념

여기서 문제가 한 번 다시 쓰였다는 점을 놓치지 마십시오. 원래 데이터가 준 것은 RUL 이라는 **연속된 숫자**입니다. 우리는 그것을 그대로 맞히는 대신, RUL 을 기준으로 **0 과 1 이라는 두 묶음**을 만들어 그 묶음을 맞히기로 했습니다. 어려운 회귀 문제를 쉬운 분류 문제로 바꿔 쓴 것이고, 그 다시 쓰기가 바로 4장에서 만들 `failure_soon` 입니다.

개념 이진 분류와 0/1 라벨 찾고 싶은 쪽에 1 을 준다

정의 클래스가 둘뿐인 분류를 **이진 분류**라고 합니다. 관례상 **찾고 싶은 쪽에 1**, 나머지에 0 을 줍니다.

스팸 메일을 1, 정상 메일을 0 으로 두는 것과 같은 약속입니다. 우리 문제에서는 **곧 고장이 1** 이고, 1 장에서 이상을 1 로 둔 것도 같은 이유였습니다.

```
# 26_03 소리 데이터 → anomaly      1 = 이상  0 = 정상
# 27_02 엔진 데이터 → failure_soon 1 = 곧 고장 0 = 아직 괜찮음
```

왜 쓰나 찾고 싶은 쪽을 1 로 두면 **합계가 곧 찾은 건수**가 됩니다. `df["failure_soon"].sum()` 이 그대로 고장 임박 시점의 개수입니다.

응용 평가 지표도 이 약속 위에서 있습니다. **재현율**은 실제 1 중 몇 개를 잡았나, **정밀도**는 1 이라고 답한 것 중 몇 개가 맞았나입니다. 1 이 어느 쪽인지 헷갈리면 두 지표의 뜻이 통째로 뒤집힙니다.

주의

글자로 곧고장·괜찮음이라고 적어 두면 사람은 읽기 편하지만 계산이 번거로워집니다. **계산용 0/1 과 표시용 글자를 둘 다 만들어 두는 것**이 1 장에서 `anomaly` 와 `label_name` 을 함께 만든 이유였습니다.

우리 데이터에는 RUL 이라는 숫자는 있지만 **곧 고장이나 아니냐**라는 예/아니오 정답은 없습니다. 지도학습은 정답이 찍지어진 데이터로 배우는 방식이므로, 정답이 없으면 무엇을 향해 학습할지 알 수 없습니다. 그래서 4 장에서 할 일은 **정답을 직접 만들어 붙이는** 것입니다.

4.1 파생 라벨 — 있는 컬럼에서 새 정답을 끌어내기

개념 파생 라벨 (derived label) 기존 컬럼에서 규칙으로 만든 새 정답

정의 원래 데이터에 없던 컬럼을, **이미 있는 컬럼에 규칙을 적용해** 만들어 낸 것입니다. 파생은 무언가로부터 끌어낸다는 뜻입니다.

낯선 말 같지만 일상에서 늘 하는 일입니다. 아래 세 가지는 전부 같은 구조입니다.

```
# 시험 점수 → 합격 여부
합격 = (점수 >= 60)          # 경계가 되는 값: 60 점

# 기온 → 더움 여부
더움 = (기온 > 28)          # 경계가 되는 값: 28 도

# RUL → 곧 고장 여부
failure_soon = (RUL <= 30)   # 경계가 되는 값: 30 cycle
```

셋 다 **연속된 숫자를 경계 하나로 갈라 둘로 접는** 일입니다. 그 경계를 임계값(threshold)이라고 부릅니다.

왜 쓰나 회귀 문제를 분류 문제로 바꿔 쓰는 것이 바로 이 작업입니다. RUL 47 을 정확히 맞히는 것은 어렵지만, RUL 이 30 이하인가를 맞히는 것은 훨씬 쉽습니다.

메서드 Series.astype(int) 참/거짓을 1/0 정수로

정의 Series 의 자료형을 바꿉니다. 조건식이 만든 True/False 에 적용하면 **참은 1, 거짓은 0** 이 됩니다.

형태 df["새컬럼"] = (조건식).astype(int)

이 한 줄 안에서 두 단계가 일어납니다. 먼저 괄호 안의 조건식이 행마다 참/거짓을 판정하고, 그다음 astype(int) 가 그것을 숫자로 바꿉니다.

```
df["failure_soon"] = (df["RUL"] <= 30).astype(int)

print(df[["RUL", "failure_soon"]].head(3).to_string())
print("...")
print(df[["RUL", "failure_soon"]].tail(3).to_string())
```

	RUL	failure_soon
▶ 0	146	0
▶ 1	145	0
▶ 2	144	0
▶ ...		
▶	RUL	failure_soon
▶ 3611	2	1
▶ 3612	1	1
▶ 3613	0	1

앞쪽 행은 RUL 이 넉넉해 0 이고, 마지막 행들은 고장 직전이라 1 입니다. 규칙이 제대로 붙었습니다.

왜 쓰나 컴퓨터는 숫자를 다루는 데 능하고, 합계 = 곧 고장 건수가 되며, 예측 == 실제 한 줄로 채점이 됩니다. 글자로 곧고장·괜찮음이라고 적어 두면 이 중 어느 것도 편하게 되지 않습니다.

응용 `astype(int)` 없이 조건식만 넣어도 컬럼은 만들어집니다. 다만 자료형이 `bool` 이 되어 `True/False` 로 저장됩니다. 대부분의 계산은 그래도 동작하지만, 출력이 지저분하고 일부 라이브러리가 까다롭게 굴 수 있으니 **정수로 통일해 두는 편이 안전합니다.**

주의

경계값 자신이 어느 쪽에 속하는지 늘 확인하십시오. `<=` 는 30 을 포함하고 `<` 는 포함하지 않습니다. 한 글자 차이로 수백 행의 라벨이 바뀝니다.

4.2 경계값 점검 — 30 은 어느 쪽인가

임계값을 정했으면 **그 값 자신이 어느 쪽으로 들어갔는지** 반드시 확인해야 합니다. 우리 규칙은 `RUL <= 30` 이므로 정확히 30 인 행은 1 이 되어야 합니다.

```
print("RUL 이 정확히 30 인 행:", int((df["RUL"] == 30).sum()), "개")
print(df.loc[df["RUL"] == 30, ["unit_id", "RUL", "failure_soon"]].head(3).to_string())
```

```
▶ RUL 이 정확히 30 인 행: 20 개
▶      unit_id  RUL  failure_soon
▶ 116         1   30             1
▶ 318         2   30             1
▶ 515         3   30             1
```

20 개 행이 정확히 RUL 30 이고 **전부 1** 입니다. 엔진이 20 대이니 각 엔진마다 딱 한 번씩 RUL 30 인 시점을 지나는 셈입니다. 예상과 맞습니다. 이런 확인은 몇 초면 끝나는데, 건너뛰면 나중에 결과가 미묘하게 어긋난 원인을 찾으려 몇 시간을 씁니다.

4.3 클래스 분포 — 우리 데이터는 기울어져 있다

메서드 `value_counts(normalize=True)` 클래스별 개수와 비율

정의 각 값이 몇 번 나왔는지 셉니다. `normalize=True` 를 주면 **개수 대신 비율**로 돌려줍니다.

형태 `df["컬럼"].value_counts()` · `df["컬럼"].value_counts(normalize=True)`

20 일차에 배운 도구지만, 오늘은 이것으로 **클래스 불균형**이라는 새 개념을 확인합니다.

```
print(df["failure_soon"].value_counts().to_dict())
print(df["failure_soon"].value_counts(normalize=True).round(4).to_dict())
```

```
▶ {0: 2994, 1: 620}
▶ {0: 0.8284, 1: 0.1716}
```

0 이 2,994 개(82.8%), 1 이 620 개(17.2%)입니다. 우리가 정작 찾고 싶은 쪽이 적은 쪽입니다. 고장 임박 시점은 전체 운전 기간 중 일부뿐이니 당연한 결과지만, 이 기울어짐이 뒤에서 계속 문제가 됩니다.

왜 쓰나 개수와 비율을 둘 다 보는 이유는 서로 다른 것을 알려 주기 때문입니다. **개수**는 소수 클래스의 절대량이 학습에 충분한지 알려 주고(620 개면 부족하지 않습니다), **비율**은 뒤에서 `stratify` 가 지켜야 할 목표값이 됩니다.

개념 클래스 불균형 (`class imbalance`) 한쪽이 훨씬 많은 상태

정의 한 클래스가 다른 클래스보다 **훨씬 많은 상태**입니다. 많은 쪽을 다수 클래스, 적은 쪽을 소수 클래스라고 부릅니다.

왜 문제가 되는지는 숫자 하나로 드러납니다. 아무것도 배우지 않고 **무조건 0 이라고만 답하는 모델**을 만들어 보겠습니다.

```
# 아무것도 학습하지 않고 전부 0 이라고 답하는 모델
import numpy as np
```

```
naive = np.zeros(len(df), dtype=int)
print("정답률:", round((naive == df["failure_soon"]).mean() * 100, 1), "%")
print("잡아낸 고장 임박:", int(((naive == 1) & (df["failure_soon"] == 1)).sum()), "건")
```

- ▶ 정답률: 82.8 %
- ▶ 잡아낸 고장 임박: 0 건

정답률 82.8%인데 고장은 단 한 건도 못 잡았습니다. 겉보기 숫자는 훌륭한데 속은 완전히 비어 있습니다. 예방 정비라는 본래 목적에서 보면 이 모델의 가치는 0 입니다.

왜 쓰나 그래서 불균형 데이터에서는 **전체 정답률을 믿으면 안 됩니다**. 1 장에서 교차표를 본 이유와 같습니다. 소수 클래스를 얼마나 잡았는지를 따로 봐야 합니다.

자주 하는 실수

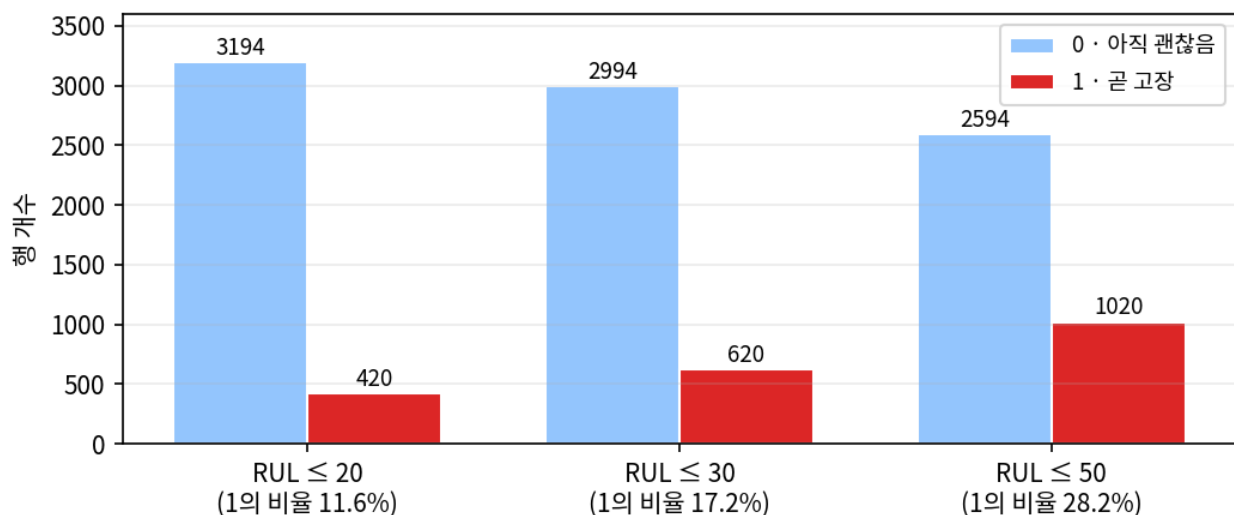
불균형은 나눌 때도 사고를 냅니다. 620 개가 3,614 개 사이에 흩어져 있는데 무작위로 20%를 떼면, 운이 나쁠 경우 **평가용에 1 이 지나치게 적게 들어갑니다**. 6 장의 stratify 가 바로 이 사고를 막는 장치입니다.

4.4 임계값을 바꾸면 무슨 일이 일어나나

임계값 30 은 데이터가 준 값이 아니라 우리가 정한 값입니다. 그러면 다른 값으로 바꾸면 어떻게 되는지 직접 확인해 봐야 합니다.

```
for t in [20, 30, 50]:
    mask = (df["RUL"] <= t).astype(int)
    print(f"임계값 {t:2d} → 1: {int(mask.sum()):4d}개 ({mask.mean()*100:.1f}%)"
```

- ▶ 임계값 20 → 1: 420 개 (11.6%)
- ▶ 임계값 30 → 1: 620 개 (17.2%)
- ▶ 임계값 50 → 1: 1020 개 (28.2%)



▲ 같은 데이터, 같은 3,614 행인데 임계값만 바꾸면 정답이 420 개에서 1,020 개로 두 배 넘게 변합니다 — 라벨은 데이터가 아니라 선택입니다

같은 데이터인데 정답이 완전히 달라집니다. 임계값을 50 으로 올리면 더 일찍, 더 자주 경고합니다. 농칠 위험은 줄지만 헛점검이 늘어납니다. 20 으로 내리면 임박한 것만 경고합니다. 점검 부담은 줄지만 농칠 위험이 커집니다. 2 장에서 본 거짓 경보와 농침의 저울이 여기서 다시 나옵니다.

기준	임계값	결과	어떤 현장에
보수적	50 (높임)	1 이 1,020 개. 미리 많이 경고합니다.	멈추면 라인 전체가 서는 설비 · 부품 조달이 오래 걸리는 설비
표준	30	1 이 620 개. 이번 수업의 선택입니다.	—
민감	20 (낮춤)	1 이 420 개. 임박한 것만 경고합니다.	여분이 있는 보조 설비 · 점검 인력이 빠듯한 현장

설비 적용

임계값을 정하는 실전 기준은 **정비 리드타임**입니다. 부품 조달에 2 주가 걸린다면 적어도 2 주 전에는 경고가 떠야 합니다. cycle 을 시간으로 환산해 조달 기간을 덮을 수 있는 값을 고르는 것이 근거 있는 선택입니다. 30 이라는 숫자를 외우는 것이 아니라, **왜 30 인지 설명할 수 있는 것**이 분석가의 일입니다.

4.5 경계 부근은 왜 어려운가

임계값 30 은 라벨을 **딱 끊어** 놓습니다. RUL 30 은 1 이고 31 은 0 입니다. 그런데 센서값은 그렇게 끊기지 않습니다. **RUL 30 일 때와 31 일 때 엔진 상태가 갑자기 달라질 리가 없습니다.** 이 어긋남이 분류 문제의 근본적인 난점이고, 다음 시간 모델이 100%를 못 맞히는 이유이기도 합니다.

숫자로 확인해 보겠습니다. 경계 바로 근처(RUL 26~34)의 행들만 뽑아 0 그룹과 1 그룹의 센서 평균을 비교하고, 그 차이를 전체 표준편차로 나눠 보았습니다. 1 에 가까울수록 두 그룹이 뚜렷이 갈린다는 뜻입니다.

```
band = df[(df["RUL"] >= 26) & (df["RUL"] <= 34)]
print("경계 구간 행 수:", len(band), " · 1:", int(band["failure_soon"].sum()))

# 두 그룹의 센서 평균 차이를 전체 표준편차로 나눠 비교
g_band = band.groupby("failure_soon")[feats].mean()
g_all = df.groupby("failure_soon")[feats].mean()
std = df[feats].std()

print("경계 구간:", ((g_band.loc[1] - g_band.loc[0]).abs() / std).round(2).to_dict())
print("전체 :", ((g_all.loc[1] - g_all.loc[0]).abs() / std).round(2).to_dict())
```

- ▶ 경계 구간 행 수: 180 · 1: 100
- ▶ 경계 구간: {'sensor_2': 0.04, 'sensor_3': 0.09, 'sensor_4': 0.2, 'sensor_7': 0.24, 'sensor_11': 0.19, 'sensor_15': 0.16}
- ▶ 전체 : {'sensor_2': 1.43, 'sensor_3': 1.04, 'sensor_4': 1.38, 'sensor_7': 1.54, 'sensor_11': 1.41, 'sensor_15': 1.5}

차이가 확연합니다. **데이터 전체로 보면** 0 그룹과 1 그룹의 센서 평균은 1.0 에서 1.5 표준편차만큼 벌어져 있습니다. 충분히 갈라낼 수 있는 간격입니다. 그런데 **경계 부근 180 행만 떼어 보면** 그 차이가 0.04 에서 0.24 로 주저앉습니다. 라벨은 0 과 1 로 확실히 갈려 있는데 **센서값으로는 사실상 구분이 안 됩니다.**

보는 범위	두 그룹 센서 평균 차이	뜻
전체 3,614 행	1.04 ~ 1.54 표준편차	확실히 갈립니다. 모델이 배울 것이 있습니다.
경계 구간 180 행 (RUL 26~34)	0.04 ~ 0.24 표준편차	거의 겹칩니다. 무엇을 보고 갈라야 할지 단서가 없습니다.

개념

이것이 2 장에서 본 **회색 지대**와 정확히 같은 이야기입니다. 소리 데이터에서 정상과 이상이 섞여 있던 가운데 구역, 엔진 데이터에서 RUL 30 근처의 180 행 — 둘 다 **정답은 갈려 있는데 입력은 안 갈리는 자리**입니다. 거짓 경보와 놓침은 대부분 여기서 나옵니다. 모델을 아무리 잘 만들어도 이 구간은 남습니다.

설비 적용

그래서 실무에서는 **경계 부근의 예측을 그대로 믿지 않습니다**. 다음 시간에 배울 `predict_proba`가 돌려주는 확률이 0.5 근처인 행들이 바로 이 구간입니다. 확률이 0.9 인 행은 바로 점검을 걸고, 0.5 근처인 행은 사람이 한 번 더 보거나 다음 주기까지 관찰하는 식으로 다르게 처리하는 것이 현실적인 운영 방식입니다.

4.6 만든 라벨을 재현 가능하게 남기기

임계값 30 은 우리가 고른 값이고, 앞 절에서 봤듯 50 으로 바꾸면 정답이 통째로 달라집니다. 그러면 몇 주 뒤에 이 노트북을 다시 열었을 때 **어떤 기준으로 만든 라벨인지**가 코드 어딘가에 남아 있어야 합니다. 숫자를 코드 중간에 그대로 박아 두면 나중에 찾기도 어렵고 바꾸기도 번거롭습니다.

```
# x 숫자를 코드 곳곳에 박아 두면
df["failure_soon"] = (df["RUL"] <= 30).astype(int)
band = df[(df["RUL"] >= 26) & (df["RUL"] <= 34)]          # 30 을 또 손으로 계산

# ✓ 위쪽에 한 번만 정의하고 이름을 붙여 두면
THRESHOLD = 30                                           # 정비 리드타임 2 주 기준

df["failure_soon"] = (df["RUL"] <= THRESHOLD).astype(int)
band = df[df["RUL"].between(THRESHOLD - 4, THRESHOLD + 4)]

print("임계값", THRESHOLD, " · 1 의 개수", int(df["failure_soon"].sum()))

▶ 임계값 30 · 1 의 개수 620
```

이렇게 두면 임계값을 50 으로 바꿔 보는 실험이 **한 줄 수정**으로 끝납니다. 주석에 왜 그 값인지까지 적어 두면 나중에 검토하는 사람도 근거를 알 수 있습니다. 4.4 에서 임계값 세 개를 비교한 실험을 반복문으로 돌릴 수 있었던 것도 값이 변수로 빠져 있었기 때문입니다.

설비 적용

라벨을 붙인 데이터는 `to_csv()`로 저장해 두는 편이 좋습니다. **라벨은 데이터가 아니라 판단의 결과**이므로, 언제 어떤 기준으로 만든 것인지 함께 남겨야 합니다. 파일 이름에 임계값을 넣거나(`labeled_th30.csv`), 컬럼 하나를 더 두어 기준을 기록해 두면 몇 달 뒤에도 되짚을 수 있습니다. 프로젝트에서 결과가 어긋났을 때 가장 먼저 확인하게 되는 것이 이 기록입니다.

라벨을 만들었으니 이제 표를 둘로 갈라야 합니다. **모델에게 보여 줄 것과 맞히라고 시킬 것**입니다. 갈라 두지 않으면 모델은 어디까지 보고 어디부터 맞혀야 하는지 알 수 없고, scikit-learn 의 도구들도 애초에 둘을 따로 받도록 설계돼 있습니다.

5.1 feature 와 target

개념 feature (X) · target (y) 입력 단서와 맞힐 정답

정의 feature 는 모델에게 보여 주는 입력 단서들로 보통 대문자 X로 씁니다. target 은 모델이 맞혀야 할 정답으로 소문자 y로 씁니다.

의사에 빗대면 이해가 빠릅니다. 체온·혈압·검사 수치가 feature 이고, 그것을 보고 내리는 진단이 target 입니다. 좋은 feature 는 **결과를 알려 주는 단서**여야 합니다.

```
feats = ["sensor_2", "sensor_3", "sensor_4",
         "sensor_7", "sensor_11", "sensor_15"]
```

```
X = df[feats]          # 입력 — 센서 6 개
y = df["failure_soon"] # 정답 — 0 또는 1
```

```
print("X:", X.shape)
print("y:", y.shape)
```

```
▶ X: (3614, 6)
▶ y: (3614,)
```

X는 3,614 행 6 열의 표이고 y는 3,614 개짜리 한 줄입니다. **행 개수가 같아야 합니다**. 입력 한 줄마다 정답 한 개가 짝지어져야 하기 때문입니다. 여기서 숫자가 어긋나면 그 뒤 모든 단계가 무너집니다.

왜 쓰나 대문자 X와 소문자 y는 단순한 관습이 아니라 **모양의 차이**를 반영합니다. X는 여러 열을 가진 2차원 표 대문자, y는 한 줄짜리 1차원이라 소문자입니다. scikit-learn 문서 어디를 열어도 이 표기가 나오므로 익혀 두면 남의 코드를 읽기가 쉬워집니다.

주의

X = df[feats]처럼 **리스트로 감싸야** 데이터프레임이 됩니다. df["sensor_2"]처럼 문자열 하나만 주면 Series가 되어 열이 하나뿐인 입력이 됩니다. 대괄호 개수 하나 차이입니다.

5.2 X에서 빼야 하는 컬럼들

9개 컬럼 중 X에 들어간 것은 센서 6개뿐입니다. 나머지 셋은 각각 다른 이유로 빼졌습니다.

컬럼	왜 빼나	넣으면
unit_id	설비 번호일 뿐 고장의 단서가 아닙니다. 그냥 이름표입니다.	16번 엔진이 오래 산다 같은 엉뚱한 규칙을 배웁니다.
cycle	3장에서 봤듯 설비마다 의미가 다릅니다. 50 cycle이 어느 엔진에는 초반이고 어느 엔진에는 중반입니다.	일관되지 않은 단위로 판단하게 됩니다.
RUL	정답을 그대로 알려 주는 값입니다. 정답 누수 입니다.	학습은 만점, 실전은 무력해집니다.

5.3 정답 누수 — 시험지에 답을 적어 둔 셈

개념 정답 누수 (data leakage) 정답 정보가 입력에 섞여 드는 사고

정의 모델이 실제로는 알 수 없어야 할 **정답 정보가 입력에 섞여 드는 일**입니다. 학습이 아니라 커닝이 됩니다.

RUL 이 왜 누수인지는 말로 설명하는 것보다 직접 해 보는 편이 빠릅니다. **모델 없이 RUL** 하나만 가지고 정답을 되살려 보겠습니다.

```
# 모델도, 학습도 없이 RUL 만으로 정답을 다시 계산
recalc = (df["RUL"] <= 30).astype(int)

# 되살린 라벨이 원래 정답과 전부 같은가?
print((recalc == df["failure_soon"]).all())
print("일치한 행:", int((recalc == df["failure_soon"]).sum()), "/", len(df))
```

▶ True
▶ 일치한 행: 3614 / 3614

3,614 행 전부 일치합니다. 학습도, 센서도, 모델도 필요 없었습니다. RUL 하나면 정답이 100% 결정됩니다. 당연합니다 — 애초에 failure_soon 을 RUL 로 만들었으니까요.

왜 쓰나 그러니 RUL 을 X 에 넣으면 모델은 센서를 볼 이유가 없습니다. RUL 만 보고 규칙 한 줄을 외워 버립니다. 성적표에는 정답률 100%가 찍히는데, **정작 현장에서는 RUL 을 모릅니다.** 그것을 알아내려고 모델을 만드는 것이니까요. 학습은 만점, 실전은 무력입니다.

응용 누수는 이렇게 뻔한 형태로만 오지 않습니다. 실무에서 자주 나오는 것은 **미래 정보가 섞여 드는 경우**입니다. 정비 완료일이나 교체 부품 코드처럼 고장이 난 뒤에야 기록되는 컬럼을 입력에 넣으면, 모델은 그것으로 답을 맞히고 실전에서는 무너집니다. **이 값을 예측 시점에 정말 알 수 있는가**를 컬럼마다 물어보는 것이 유일한 방어입니다.

개념

.all()은 Series 의 값이 **전부 참인지** 한 번에 확인합니다. 하나라도 거짓이면 False 가 됩니다. 짝이 되는 .any()는 **하나라도 참이면 True** 를 돌려줍니다. 검증 코드에서 자주 쓰이는 짝입니다.

설비 적용

프로젝트 리포트를 쓸 때 **입력에 쓴 컬럼과 뺀 컬럼, 그리고 뺀 이유**를 한 줄씩 적어 두십시오. 정답률이 유난히 높게 나왔을 때 가장 먼저 의심할 곳이 이 목록이고, 검토하는 사람도 그 목록부터 봅니다.

주의

feature 는 많다고 좋은 것이 아닙니다. 관계없는 컬럼이 섞이면 모델이 엉뚱한 패턴을 찾아 헤매고, 너무 적으면 판단할 근거가 부족합니다. 이번 수업은 **변화가 뚜렷한 센서 6 개로 미리 골라 둔 것**을 그대로 씁니다. feature 를 잘 고르는 일은 그 자체로 큰 주제라 나중에 따로 다룹니다.

오늘 수업에서 **가장 오래 붙잡고 있었던 대목**입니다. 데이터가 준비됐으니 이제 전부 모델에 넣으면 될 것 같지만, 그러면 그 모델의 실력을 잴 방법이 사라집니다. 배운 문제를 그대로 다시 푸는 것은 시험이 아니기 때문입니다.

그래서 데이터를 두 몫으로 나눕니다. **학습용은 가르치는 데 쓰고, 평가용은 다 배운 뒤 실력을 잴 때 단 한 번만 씁니다.** 평가용을 학습에 끼워 넣으면 성적이 부풀려지고, 현장 성능을 잘못 예상하게 됩니다. 이 약속을 지켜야 평가가 공정해집니다.

6.1 train_test_split — 네 덩어리로 쪼개기

함수 train_test_split() 학습용과 평가용으로 나누기

정의 X와 y를 넣으면 네 덩어리로 나눠 돌려줍니다. 나누기 전에 무작위로 섞습니다.

형태 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=..., random_state=..., stratify=...)

scikit-learn의 model_selection에 들어 있습니다. 먼저 불러옵니다.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print("X_train:", X_train.shape, " y_train:", y_train.shape)
print("X_test :", X_test.shape, " y_test :", y_test.shape)
```

▶ X_train: (2891, 6) y_train: (2891,)
▶ X_test : (723, 6) y_test : (723,)

3,614 행이 **2,891 행과 723 행**으로 갈렸습니다. $723 \div 3,614 = 0.2$ 로 정확히 20%입니다. 열은 양쪽 다 6 개 그대로이고, 정답도 같은 비율로 따라 나뉘었습니다.

왜 쓰나 네 덩어리의 이름 끝만 보면 됩니다. train이 붙으면 가르칠 때 쓰고, test가 붙으면 시험 볼 때 씁니다. X는 입력, y는 정답입니다.

자주 하는 실수

반환 순서는 고정입니다. X_train, X_test, y_train, y_test 순서로 나옵니다. 왼쪽 변수 이름을 어떻게 바꿔 적든 들어가는 내용은 순서대로 정해집니다. X_train, y_train, X_test, y_test 처럼 순서를 착각해 받으면 입력 자리에 정답이 들어가는데, 오류 없이 실행되다가 뒤에서 이상한 결과로 나타납니다.

개념 일반화와 과적합 외운 것과 이해한 것의 차이

정의 일반화는 처음 보는 데이터에서도 맞는 능력입니다. 학습 데이터만 잘 맞히고 새 데이터에 약한 상태를 **과적합**이라고 합니다. 문제집을 통째로 외운 학생과 원리를 이해한 학생을 생각하면 됩니다. 외운 학생은 본 문제는 다 맞히지만 조금만 비틀면 무너집니다. 모델도 똑같습니다. 그래서 **본 적 없는 문제로 시험을 봐야** 실력을 알 수 있습니다.

```
# 다음 시간에 이렇게 비교하게 됩니다
# train_score = model.score(X_train, y_train) # 배운 문제
# test_score = model.score(X_test, y_test) # 처음 보는 문제

# 두 값이 비슷하면 → 일반화가 잘 된 것
# 학습은 높고 평가가 낮으면 → 외운 것 (과적합 의심)
```

오늘 데이터를 나눠 둔 이유가 바로 이 비교를 하기 위해서입니다. 나누지 않았다면 배운 문제를 다시 푸는 것이라 **성적이 아무 의미가 없습니다.**

왜 쓰나 평가용을 학습에 한 번이라도 끼워 넣으면 성적이 부풀려지고, 그 숫자를 믿고 현장에 내보냈다가 성능이 안 나오는 사고가 납니다. **평가용은 다 배운 뒤 단 한 번만** 쓴다는 약속을 지켜야 합니다.

자주 하는 실수

평가용을 여러 번 보면서 옵션을 조금씩 바꿔 성적을 올리는 것도 넓은 의미의 누수입니다. 평가용에 맞춰 튜닝한 셈이라 **그 성적은 더 이상 처음 보는 데이터의 성적이 아닙니다.** 실무에서는 이런 경우 검증용을 따로 하나 더 떼어 셋으로 나눕니다.

6.2 세 개의 옵션

위 코드에서 옵션 세 개를 함께 넘겼습니다. 각각이 무엇을 결정하는지 하나씩 봅시다.

문법 `test_size` 평가용으로 떼어 둘 비율

정의 전체 중 **평가용으로 떼어 둘 비율**입니다. 0.2 면 20%가 평가용, 나머지 80%가 학습용이 됩니다.

값을 바꾸면 두 몫의 크기가 어떻게 변하는지 직접 확인해 보겠습니다.

```
X_tr3, X_te3, y_tr3, y_te3 = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

print("test_size 0.2 → 학습용", X_train.shape[0], "· 평가용", X_test.shape[0])
print("test_size 0.3 → 학습용", X_tr3.shape[0], "· 평가용", X_te3.shape[0])
```

- ▶ `test_size 0.2` → 학습용 2891 · 평가용 723
- ▶ `test_size 0.3` → 학습용 2529 · 평가용 1085

평가용을 30%로 늘리니 학습용이 2,891 에서 **2,529 로 362 행 줄었습니다.** 시험은 더 정밀해지지만 가르칠 자료가 줄어듭니다.

왜 쓰나 둘 사이의 균형입니다. 평가용을 너무 크게 잡으면(0.4 등) 시험은 믿을 만해도 학습이 부실해지고, 너무 작게 잡으면(0.1 등) 학습은 넉넉해도 시험 결과를 믿기 어렵습니다. **통상 0.2 에서 0.3 사이**를 쓰고, 이번 수업은 0.2 입니다.

문법 `random_state` 같은 결과를 다시 만들기

정의 섞는 방식을 고정하는 숫자입니다. **같은 값을 주면 누가 언제 실행해도 똑같이 나옵니다.**

값을 주지 않으면 실행할 때마다 다르게 섞입니다. 그러면 어제 낸 결과를 오늘 재현할 수 없고, 강사와 수강생의 결과도 어긋납니다.

```
a = train_test_split(X, y, test_size=0.2, random_state=42)[3]
b = train_test_split(X, y, test_size=0.2, random_state=42)[3]
c = train_test_split(X, y, test_size=0.2, random_state=7)[3]

print("42 두 번 → 같은가?", (a.index == b.index).all())
print("42 vs 7 → 같은가?", (a.index == c.index).all())
```

- ▶ `42 두 번 → 같은가?` True
- ▶ `42 vs 7 → 같은가?` False

같은 값이면 완전히 같은 분할이 나오고, 값이 다르면 다른 분할이 나옵니다.

왜 쓰나 재현성 때문입니다. 리포트에 적은 숫자를 나중에 누가 다시 돌려도 똑같이 나와야 신뢰할 수 있습니다. 결과가 이상할 때 원인을 좁혀 갈 수도 있습니다.

개념

42 라는 숫자 자체에 특별한 의미는 없습니다. **그냥 약속입니다.** 0 이든 7 이든 2026 이든 상관없고, 팀 안에서 하나로 통일해 두기만 하면 됩니다. 어제 `IsolationForest` 에 넘긴 `random_state=42` 도 같은 성격의 값이었습니다.

6.3 stratify — 기울어진 비율을 양쪽에 그대로

4장에서 확인한 대로 우리 데이터는 0이 82.8%, 1이 17.2%로 기울어져 있습니다. 620 개의 1이 3,614 행 사이에 흩어져 있는데 무작위로 20%를 뚫 떼면 어떻게 될까요. **운이 나쁘면 평가용에 1이 지나치게 적게 들어갑니다.** 그러면 그 시험은 소수 클래스를 제대로 재지 못합니다.

문법 `stratify=y` 원래의 클래스 비율을 양쪽에 유지

정의 넘겨준 라벨의 클래스 비율을 학습용과 평가용 양쪽에 그대로 유지하면서 나눕니다. 불균형 데이터에서 특히 중요합니다.

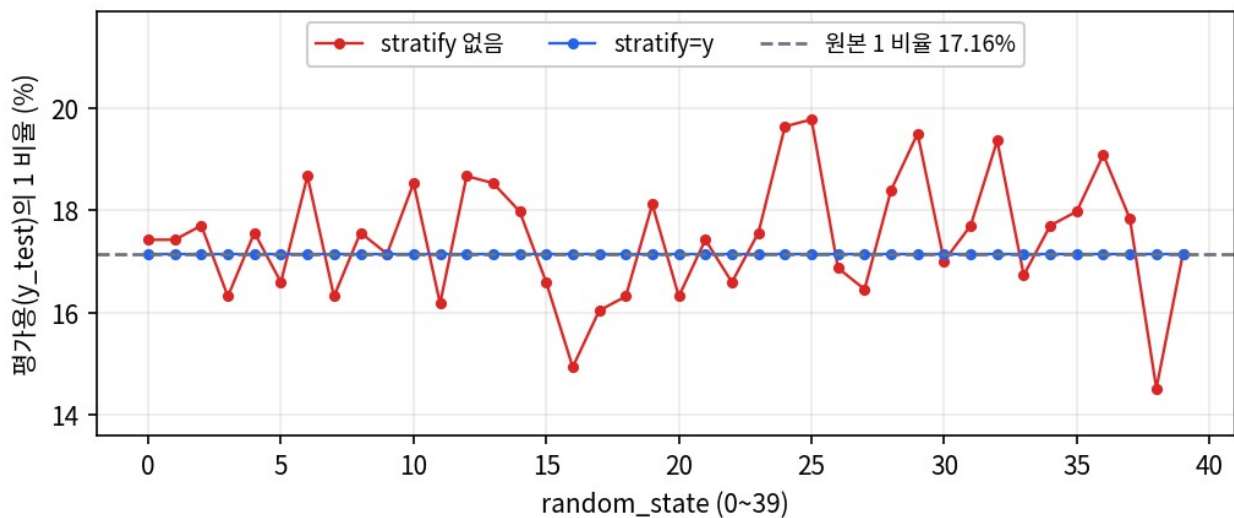
형태 `train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`

제대로 지켜졌는지 확인하는 것이 이 절의 핵심입니다. 양쪽의 비율을 각각 출력해 원본과 비교합니다.

```
print("원본 :", df["failure_soon"].value_counts(normalize=True).round(6).to_dict())
print("학습용 :", y_train.value_counts(normalize=True).round(6).to_dict())
print("평가용 :", y_test.value_counts(normalize=True).round(6).to_dict())
```

- ▶ 원본 : {0: 0.828445, 1: 0.171555}
- ▶ 학습용 : {0: 0.828433, 1: 0.171567}
- ▶ 평가용 : {0: 0.828492, 1: 0.171508}

소수점 셋째 자리까지 세 값이 같습니다. 0.8284와 0.1716이 원본·학습용·평가용에 그대로 유지됐습니다. 미세한 차이는 620 개를 정확히 5로 나눌 수 없어 생기는 반올림일 뿐입니다. `stratify`가 제대로 작동했다는 증거입니다.



▲ `random_state`를 0부터 39까지 바꿔 가며 평가용의 1 비율을 켜 것 — 파란 선(`stratify=y`)은 회색 기준선에 딱 붙어 미동도 없고, 빨간 선(`stratify 없음`)은 14.5%에서 19.8%까지 널을 뒀습니다

왜 쓰나 이 확인 자체가 결과물입니다. **분리가 제대로 됐다는 것을 숫자로 입증해 두면**, 나중에 모델 성능이 이상하게 나왔을 때 분리 단계는 용의선상에서 지울 수 있습니다. 프로젝트 리포트에는 이 비교를 꼭 넣으십시오.

응용 효과가 얼마나 큰지 보려면 `stratify`를 뺀 것과 나란히 놓아 보면 됩니다. `random_state`를 0부터 39까지 바꿔 가며 40번 나눠 보고, 평가용의 1 비율이 어떻게 흔들리는지 재 봤습니다.

```
import numpy as np

no_s, with_s = [], []
for rs in range(40):
    _, _, _ , a = train_test_split(X, y, test_size=0.2, random_state=rs)
    _, _, _ , b = train_test_split(X, y, test_size=0.2, random_state=rs, stratify=y)
    no_s.append(a.mean()); with_s.append(b.mean())

no_s, with_s = np.array(no_s), np.array(with_s)
print(f"stratify 없음: {no_s.min()*100:.2f}% ~ {no_s.max()*100:.2f}% (폭 {(no_s.max()-no_s.min())*100:.2f}%p)")
print(f"stratify 있음: {with_s.min()*100:.2f}% ~ {with_s.max()*100:.2f}% (폭 {(with_s.max()-with_s.min())*100:.2f}%p)")
```

▶ stratify 없음: 14.52% ~ 19.78% (폭 5.26%p)
▶ stratify 있음: 17.15% ~ 17.15% (폭 0.00%p)

stratify 없이 나누면 평가용의 1 비율이 **14.52%에서 19.78%까지** 흔들립니다. random_state를 무엇으로 잡느냐에 따라 시험지의 난이도가 달라지는 셈입니다. stratify=y를 주면 **40번 모두 17.15%로 고정**됩니다.

설비 적용

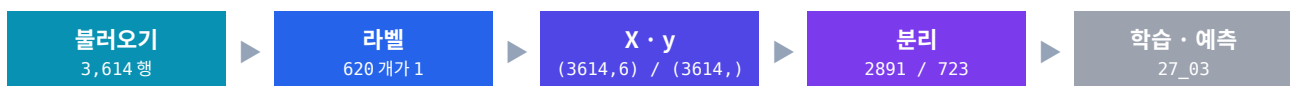
설비 데이터에서 이 문제는 훨씬 심각해집니다. 오늘은 1이 17%나 되지만, 실제 현장의 고장 데이터는 1이 1%도 안 되는 경우가 흔합니다. 그런 데이터를 무작위로 나누면 **평가용에 고장 사례가 한 건도 안 들어가는** 일이 실제로 일어납니다. 그 시험은 아무것도 재지 못합니다.

주의

stratify에는 나누고 싶은 기준이 되는 **라벨을 넘깁니다**. 보통 stratify=y입니다. stratify=True 같은 것은 없고, 넘긴 라벨 중 **한 클래스가 2개 미만이면 오류**가 납니다. 꼬꺄 수가 없기 때문입니다.

6.4 여기까지가 오늘

네 덩어리가 만들어졌습니다. 이제 X_train과 y_train을 모델에 넣어 학습시키고, X_test로 예측을 받아 y_test와 맞춰 보면 됩니다. **그것이 다음 시간입니다**.



연결

오늘의 마지막 코드가 만든 네 변수는 **다음 시간의 첫 코드가 그대로 받아 씁니다**. RandomForestClassifier를 만들어 fit(X_train, y_train)으로 가르치고, predict(X_test)로 답을 받아 y_test와 비교합니다. 오늘 준비를 제대로 해 두었기 때문에 다음 시간 코드는 몇 줄 되지 않습니다.

복습할 때는 이 표만 훑어도 됩니다. 기억이 안 나는 줄이 있으면 그 도구가 나온 장으로 돌아가십시오.

A.1 이상 라벨링과 시각화 (26_03)

도구	쓰는 법	하는 일
패턴	<code>(pred == -1).astype(int)</code>	모델의 -1/1 을 이상=1 · 정상=0 라벨로 뒤집습니다.
메서드	<code>.map({1:"이상", 0:"정상"})</code>	숫자 라벨을 그림·보고용 글자 라벨로 바꿉니다.
메서드	<code>iso.score_samples(df[feats])</code>	이상 점수를 매깁니다. 작을수록 이상합니다.
패턴	<code>df[df["anomaly"]==1].sort_values("score")</code>	이상만 추려 점점 우선순위 목록을 만듭니다.
함수	<code>pd.crosstab(정답, 모델라벨)</code>	맞힘·거짓 경보·놓침을 한 표로 채점합니다.
함수	<code>sns.scatterplot(..., hue="label_name")</code>	라벨에 따라 색이 갈리는 산점도. 범례도 자동입니다.
함수	<code>plt.axhline(y=thr, color="red", linestyle="--")</code>	임계값 위치에 가로 기준선을 긋습니다.
문법	<code>plt.scatter(..., c=score, cmap="viridis")</code>	점수를 색의 진하기로 표현해 정도를 보여 줍니다.
함수	<code>plt.savefig("f.png", dpi=150, bbox_inches="tight")</code>	그림을 파일로 저장합니다. show 보다 먼저 부릅니다.

A.2 고장 임박 분류 준비 (27_01 · 27_02)

도구	쓰는 법	하는 일
메서드	<code>df["unit_id"].nunique()</code>	서로 다른 값의 개수. 설비 대수를 셉니다.
패턴	<code>df.groupby("unit_id")["cycle"].max()</code>	설비별 최대 cycle = 그 설비의 수명입니다.
메서드	<code>(df["RUL"] <= 30).astype(int)</code>	참/거짓을 1/0 으로 바꿔 파생 라벨을 만듭니다.
메서드	<code>.value_counts(normalize=True)</code>	클래스별 비율. 불균형을 확인합니다.
메서드	<code>(recalc == df["failure_soon"]).all()</code>	전부 같은지 한 번에 확인합니다. 누수 검증에 씁니다.
패턴	<code>X = df[feats] · y = df["failure_soon"]</code>	입력과 정답을 나눠 담습니다. 행 수가 같아야 합니다.
함수	<code>train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)</code>	학습용·평가용 네 덩어리로 나눕니다.

A.3 오늘의 숫자

데이터	항목	값
MIMII 소리 (220 행)	IsolationForest 가 잡은 이상	40 건 (contamination 0.18)
	교차표 — 정상 맞힘 · 거짓 경보 · 놓침 · 이상 맞힘	166 · 14 · 14 · 26
	재현율 · 정밀도	65.0% · 65.0%
	rms 평균+3 시그마 기준선 · 선 위 점	0.135 · 5 개 (모델이 잡은 40 개 중 35 개는 선 아래)
C-MAPSS 엔진 (3,614 행)	설비 대수 · 수명 범위	20 대 · 147 ~ 215 cycle

데이터	항목	값
	failure_soon 0 / 1	2,994 / 620 (82.8% / 17.2%)
	임계값 20 · 30 · 50 일 때 1 의 개수	420 · 620 · 1,020
	분리 결과 (test_size 0.2)	학습용 2,891 · 평가용 723
	stratify 유무에 따른 평가용 1 비율 (40 회)	없음 14.52~19.78% · 있음 17.15% 고정

A.4 확인용 코드 모음

결과가 이상할 때 원인을 좁히는 데 쓰는 짧은 코드들입니다. 오늘 각 장에서 실제로 썼던 확인 방법을 모았습니다.

```
# 라벨 방향이 뒤집히지 않았는가 (1 장)
print(int(df["anomaly"].sum())) # contamination 비율과 맞는가

# 변환에서 조용히 결측이 생기지 않았는가 (1 장)
print(df["label_name"].value_counts().sum() == len(df))

# 경계값이 어느 쪽에 들어갔는가 (4 장)
print(df.loc[df["RUL"] == 30, "failure_soon"].unique()) # [1] 이어야 함

# 입력에 정답이 섞이지 않았는가 (5 장)
print("RUL" in X.columns, "cycle" in X.columns) # 둘 다 False

# 입력과 정답의 행 수가 맞는가 (5 장)
print(X.shape[0] == y.shape[0])

# 분리 비율이 의도대로인가 (6 장)
print(round(X_test.shape[0] / (X_train.shape[0] + X_test.shape[0]), 3))

# stratify가 비율을 지켰는가 (6 장)
print(y_train.value_counts(normalize=True).round(4).to_dict())
print(y_test.value_counts(normalize=True).round(4).to_dict())
```

```
▶ 40
▶ True
▶ [1]
▶ False False
▶ True
▶ 0.2
▶ {0: 0.8284, 1: 0.1716}
▶ {0: 0.8285, 1: 0.1715}
```

이 여덟 줄을 노트북 마지막 셀에 모아 두면 **분리 단계까지의 준비가 제대로 됐는지 한 번에 확인할 수** 있습니다. 프로젝트에서도 데이터가 바뀔 때마다 이 셀만 다시 돌려 보면 됩니다.

B.1 오늘 코드에서 틀리기 쉬운 곳

X 이렇게 하면	무슨 일이	✓ 바로잡기
<code>(pred == 1).astype(int)</code>	정상이 1 이 되어 모든 결과가 거꾸로. 오류는 안 납니다.	<code>(pred == -1).astype(int)</code>
<code>plt.show()</code> 뒤에 <code>savefig()</code>	하얀 빈 그림이 저장됩니다. 오류도 안 납니다.	<code>savefig</code> 를 <code>show()</code> 앞에
<code>plt.colorbar()</code> 만 호출	어느 그림의 색인지 몰라 오류나 엉뚱한 막대가 붙습니다.	<code>sc = plt.scatter(...)</code> 후 <code>plt.colorbar(sc)</code>
<code>hue="anomaly"</code>	범례가 0 과 1 로 떠서 어느 쪽이 이상인지 모릅니다.	<code>hue="label_name"</code>
<code>X_train, y_train, X_test, y_test = ...</code>	입력 자리에 정답이 들어갑니다. 반환 순서는 고정입니다.	<code>X_train, X_test, y_train, y_test</code>
<code>X = df[feats + ["RUL"]]</code>	정답 누수. 학습은 100%, 실전은 무력해집니다.	<code>X = df[feats]</code> — RUL 제외
<code>train_test_split(X, y, test_size=0.2)</code>	평가용 1 비율이 14.5~19.8%로 흔들립니다.	뒤에 <code>random_state=42, stratify=y</code>
<code>(df["RUL"] < 30)</code>	RUL 이 정확히 30 인 20 개 행이 0 이 됩니다.	<code>(df["RUL"] <= 30)</code>
<code>X = df["sensor_2"]</code>	Series 가 되어 열이 하나뿐인 입력이 됩니다.	<code>X = df[["sensor_2"]]</code> — 대괄호 두 겹
<code>.map({1: "이상"})</code> — 0 을 빠뜨림	0 인 행이 조용히 NaN 이 됩니다. 오류가 안 납니다.	딕셔너리에 모든 값을 적고 개수 합을 확인

B.2 교안과 실행값이 다른 곳

26_03 교안의 교차표 슬라이드에 적힌 숫자가 실제 실행 결과와 다릅니다. 26_mimii_features.csv 를 교안과 같은 옵션 (contamination=0.18, random_state=42)으로 돌린 값을 나란히 놓았습니다. 이 책의 본문에는 실행값을 실었습니다.

항목	교안 표기	실행값	차이
정상 맞힘	167	166	-1
거짓 경보	(167 에 대응)	14	—
놓침	13	14	+1
이상 맞힘	27	26	-1
Z-score 대비 개선	23 개 → 27 개	23 개 → 26 개	개선 폭 4 → 3
rms 평균+3 시그마	약 0.134	0.135 (0.13507)	반올림 자리

교안 쪽 숫자는 합이 220 이 되긴 하므로 계산 실수라기보다 **작성 시점의 데이터 버전이 지금 파일과 달랐을 가능성**이 큼니다. 실습 노트북에 적힌 # 출력: 0.134 라는 주석도 실제로는 0.135 가 찍히니 같은 이유로 보입니다. 어느 쪽이든 **결론은 바뀌지 않습니다** — IsolationForest 가 Z-score 보다 몇 건 더 잡았고, 여전히 3 분의 1 이상을 놓치고 있다는 이야기 그대로입니다.

주의

교안의 예상 결과와 내 실행 결과가 다를 때 **먼저 의심할 것은 내 코드입니다**. 파일 경로, 옵션 값, 실행 순서를 확인하십시오. 그러고도 다르면 실행값을 본문에 쓰고 차이를 기록해 두면 됩니다. 리포트에 숫자를 옮겨 적을 때 교안에서 베끼지 말고 **직접 돌린 출력에서 가져오는 습관**이 이런 사고를 막아 줍니다.

C.1 26_03 실습 열 가지

개념 코드를 따라 친 뒤 같은 흐름을 열 번 반복한 구성입니다. 각 실습이 무엇을 확인하려는 것이었는지만 정리합니다.

#	실습	확인할 것
1	이상 라벨 만들고 붙이기	합계가 40 인가 — 방향이 뒤집히지 않았는가
2	이상 샘플 추출하기	점수가 작은 것이 맨 위로 올라왔는가
3	정답과 교차표 만들기	네 칸이 각각 무엇을 뜻하는지 말할 수 있는가
4	정상·이상 산점도	한글 제목이 □로 깨지지 않는가
5	색과 범례 다듬기	alpha 0.3·0.6·1.0 의 밀도 차이가 보이는가
6	두 특징 조합 비교	rms 가 들어간 조합과 빠진 조합에서 다른 이상이 보이는가
7	기준선 추가	선 위 점이 5 개뿐인 이유를 설명할 수 있는가
8	의심 구간 강조 (색 농담)	두 색보다 회색 지대가 잘 보이는가
9	이상 점수 히스토그램	정상 봉우리와 이상 꼬리가 구분되는가
10	이상탐지 종합 미니 리포트	요약·그림·우선순위·한계가 다 담겼는가

C.2 27_01 · 27_02 실습

27_01 은 새 데이터와 친해지는 탐색 위주였고, 27_02 는 오늘 배운 개념을 코드 한 줄씩으로 확인하는 구성이었습니다. **27_01 장과 27_02 장의 문제를 노트북으로 구성해 제출하는 것이 오늘 과제입니다.**

교안	실습 흐름	마지막에 확인한 것
27_01	불러오기 → info/describe → nunique → 한 설비 떼어 보기 → 설비별 수명 비교	같은 50 cycle 에서 1 번은 RUL 97, 2 번은 152 — cycle 만으로는 비교 불가
27_02	라벨 생성 → 경계값 점검 → 클래스 분포 → 임계값 실험 → 누수 체험 → X/y 구성 → 분리 → 비율 검증 → test_size 비교	학습용 2,891 vs 2,529 — test_size 를 키우면 가르칠 자료가 줄어듦

C.3 다음 시간 예고 — 27_03 랜덤 포레스트 고장 임박 분류

오늘 만든 `X_train·X_test·y_train·y_test` 네 덩어리를 그대로 받아 씁니다. 드디어 모델을 학습시키고 예측을 받아 보는 시간입니다.

단계	도구	하는 일
모델 생성	<code>RandomForestClassifier(n_estimators=100, random_state=42)</code>	트리 100 그루짜리 분류기를 만듭니다. 아직 빈 상자입니다.
학습	<code>model.fit(X_train, y_train)</code>	학습용만 넣습니다. 평가용은 절대 넣지 않습니다.
예측	<code>model.predict(X_test)</code>	0 과 1 로 이루어진 배열을 받습니다.
확신도	<code>model.predict_proba(X_test)</code>	각 행이 1 일 확률. 0.5 부근이 애매한 행입니다.
중요도	<code>model.feature_importances_</code>	어느 센서가 판단에 크게 쓰였는지. 우선 점검 부위로 이어집니다.
우선순위	예측·실제·확률을 한 표로 정렬	정비팀에 그대로 넘길 수 있는 목록이 됩니다.
확장	<code>27_mimii_features_sample.csv</code>	소리 데이터에도 같은 흐름을 적용합니다. 데이터만 바뀔 뿐입니다.

다음 시간 코드가 몇 줄인지 미리 보면 오늘 준비가 왜 중요했는지 실감이 납니다. 모델을 만들고 학습시키고 예측하는 것은 단 세 줄입니다.

다음 시간 미리보기 — 27_03

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)          # 오늘 만든 학습용 두 덩어리
y_pred = model.predict(X_test)      # 오늘 만든 평가용 입력

# y_pred 와 y_test 를 비교하면 성능이 나옵니다
```

세 줄로 끝나는 이유는 오늘 여섯 장에 걸쳐 준비를 끝내 두었기 때문입니다. 라벨을 만들고, 누수를 건어내고, 비율을 지켜 나눈 결과가 `X_train·y_train·X_test` 라는 이름으로 그대로 넘어갑니다. 실무에서 시간을 가장 많이 쓰는 곳이 모델이 아니라 그 앞단이라는 말이 이 대비에서 나옵니다.

연결

예고편을 미리 읽어 두면 다음 시간이 훨씬 수월합니다. 특히 오늘 4장에서 본 클래스 불균형이 내일 성능 평가에서 다시 발목을 잡습니다. 정답률만 보면 83%가 나오는데 그것이 아무 의미 없다는 사실을 오늘 이미 확인해 두었으니, 내일은 그 대신 무엇을 봐야 하는지가 자연스러운 다음 질문이 됩니다.

C.4 제출 전 점검표

오늘 과제는 27_01·27_02의 실습 문제를 노트북으로 구성해 제출하는 것입니다. 내기 전에 아래를 확인하십시오. 대부분 오류가 아니라 조용히 틀리는 항목들입니다.

	확인할 것	틀렸을 때 증상
☐	모든 셀이 위에서부터 순서대로 실행되는가 (커널 재시작 후 전체 실행)	변수가 없다는 <code>NameError</code> — 실행 순서에 의존한 코드
☐	출력이 셀 아래에 저장된 채로 저장했는가	채점자가 결과를 볼 수 없습니다.
☐	CSV 경로가 상대 경로로 맞는가	<code>FileNotFoundError</code> — 폴더 구조가 다른 환경에서
☐	한글 폰트 설정 셀을 맨 위에서 실행했는가	그래프 제목과 축이 □□□로 나옵니다.
☐	<code>savefig</code> 를 <code>show</code>	저장된 그림 파일이 하얗게 비어 있습니다.
☐	X에 <code>RUL·unit_id·cycle</code> 이 없는가	정답 누수. 성능이 비현실적으로 높게 나옵니다.
☐	<code>train_test_split</code> 의 반환을 순서대로 받았는가	입력 자리에 정답이 들어갑니다. 오류는 안 납니다.
☐	<code>random_state=42·stratify=y</code> 를 빠뜨리지 않았는가	돌릴 때마다 결과가 달라져 재현이 안 됩니다.

오늘 두 가지를 마쳤습니다. 하나는 **어제 만든 이상탐지 모델의 결과를 사람이 쓸 수 있는 형태로 바꾸는 일**이었고, 다른 하나는 **고장 임박 분류라는 새 문제를 세우고 데이터를 준비하는 일**이었습니다. 모델을 학습시키지 않았다고 해서 오늘 한 일이 가벼운 것은 아닙니다. 실무에서 시간을 가장 많이 쓰는 곳이 바로 오늘 한 준비 단계입니다.

오늘 확실히 하고 넘어갈 것 다섯 가지

#	확인 질문	대답이 안 나오면
1	predict 의 -1 을 왜 1 로 바꾸는지, 반대로 하면 무슨 일이 생기는지 설명할 수 있는가	1 장 1.1
2	라벨과 점수의 차이, 그리고 점수가 필요한 이유를 말할 수 있는가	1 장 1.2
3	rms 기준선이 잡은 5 개와 모델이 잡은 40 개의 차이를 설명할 수 있는가	2 장 2.2
4	RUL 을 X 에서 빼는 이유를 누수라는 말을 써서 설명할 수 있는가	5 장 5.3
5	stratify=y 가 없으면 무엇이 흔들리는지 숫자로 말할 수 있는가	6 장 6.3

복습 루틴 제안

오늘 저녁 20 분 — 부록 A 치트시트를 훑고, 위 다섯 질문에 소리 내어 답해 보십시오. 막히는 항목만 해당 절로 돌아가면 됩니다.

내일 수업 전 10 분 — 부록 C 의 예고편 표를 읽어 두십시오. 처음 보는 이름 다섯 개가 미리 눈에 익으면 수업 중에 코드를 따라 치는 여유가 생깁니다.

주말 한 시간 — 27_02 실습을 임계값 30 대신 **50 으로 바꿔** 처음부터 다시 돌려 보십시오. 클래스 비율이 어떻게 변하고, stratify 가 지키는 비율이 어떻게 달라지는지 직접 보면 4 장과 6 장이 한 번에 이어집니다. 프로젝트에서 임계값을 정할 때 그 감각이 그대로 쓰입니다.

설비 적용

오늘 배운 순서 — **라벨을 만들고, 입력에서 정답을 건어내고, 비율을 지켜 나눈다** — 는 어떤 예지보전 프로젝트에서도 똑같이 반복됩니다. 데이터가 소리든 진동이든 온도든, 모델이 랜덤 포레스트든 다른 무엇이든 이 앞단은 바뀌지 않습니다. 오늘 한 일이 가장 오래 쓰이는 부분입니다.